Recurrent networks: sequence models, gated memories and decoding

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

Text, speech, detector read-outs and the trajectory of a physical system are sequences: their length varies, and the order of their elements carries the meaning. A recurrent network reads a sequence one element at a time and keeps a hidden state that summarises everything read so far, applying the same weights at every step. This note starts from language models — counting, then the neural n-gram model — and builds the recurrent network, its training by backpropagation through time, and the analysis of why gradients vanish or explode over long time gaps, read as the stability theory of a dynamical system. It then derives the gated cells that cure the vanishing gradient (LSTM and GRU), assembles encoder–decoder models that map one sequence to another, and treats decoding by greedy and beam search, sequence classification, bidirectional and tree-structured networks. It ends with the limits of recurrence that motivate attention (Note 15) and with the modern return of linear recurrences.

1 Sequences and language models

A *sequence* is an ordered list $\mathbf{x}_1, \dots, \mathbf{x}_T$ whose length T changes from one example to the next. The elements may be vectors (the samples of a waveform, the readings of a sensor at successive times) or symbols from a finite *vocabulary* of size V (characters, words or pieces of words, called *tokens*). Note 1, Section 3.2, turned a document into a bag of words and observed that the representation cannot tell "dog bites man" from "man bites dog". This note keeps the order.

The chain rule. The basic sequence task is to assign a probability to a whole sequence, a *language model* when the symbols are tokens of text. Any joint distribution factorises, by the product rule of probability (Note 2, Section 1.1), into next-token conditionals:

$$p(w_1, \dots, w_T) = \prod_{t=1}^{T} p(w_t \mid w_1, \dots, w_{t-1}) = \prod_{t=1}^{T} p(w_t \mid w_{<t}). \tag{1}$$

No approximation is involved yet. The modelling problem is to represent the conditional distribution of the next token given a *context* $w_{<t}$ of any length. Two special tokens make the bookkeeping clean: every sequence starts with a *beginning-of-sequence* token `<bos>` that is given and never predicted, and ends with an *end-of-sequence* token `<eos>` that is predicted like any other. The model thereby also predicts when to stop, so that (1) is a distribution over sequences of all lengths. Each conditional is a categorical distribution over V outcomes (Note 2, Section 2): training maximises the log-likelihood, that is, minimises the next-token cross-entropy, and the model is judged by its *perplexity*, the exponential of the average negative log-likelihood per predicted token, on held-out text (Note 2, modern box of Section 2.2): $\mathrm{PP} = \exp\big(-\frac{1}{N}\sum_t \log p(w_t \mid w_{<t})\big)$ over the N predicted tokens.

Markov assumption and n-grams. The classical simplification keeps only the last $n-1$ tokens of the context,

$$p(w_t \mid w_{<t}) \approx p(w_t \mid w_{t-n+1}, \dots, w_{t-1}), \tag{2}$$

an $(n-1)$-th order Markov chain. Each conditional is estimated by counting how often each n-tuple of tokens (an n-gram) occurs in a training corpus, $\hat{p}(w \mid \text{context}) = N(\text{context}, w)/N(\text{context})$ with N the counts: the categorical maximum-likelihood estimate of Note 2, Section 2.1, once per context.

> **Worked example — 1. A bigram model and its zero**
>
> The training text consists of three sentences: "the muon decays .", "the pion decays .", "the muon stops .", each preceded by `<bos>`. The full stop plays the role of `<eos>`. With $n = 2$ the counts give
>
> $$\hat{p}(\text{the} \mid \texttt{<bos>}) = 1, \qquad \hat{p}(\text{muon} \mid \text{the}) = \tfrac{2}{3}, \qquad \hat{p}(\text{pion} \mid \text{the}) = \tfrac{1}{3},$$
> $$\hat{p}(\text{decays} \mid \text{muon}) = \hat{p}(\text{stops} \mid \text{muon}) = \tfrac{1}{2}, \qquad \hat{p}(\text{decays} \mid \text{pion}) = 1,$$
> $$\hat{p}(. \mid \text{decays}) = \hat{p}(. \mid \text{stops}) = 1.$$
>
> The sentence "the muon decays ." receives $1 \cdot \frac{2}{3} \cdot \frac{1}{2} \cdot 1 = \frac{1}{3}$ by (1); with four predicted tokens its perplexity is $(1/3)^{-1/4} = 3^{1/4} = 1.316$. The equally sensible sentence "the pion stops ." receives probability zero, because the bigram (pion, stops) never occurred. A pseudo-count of 1 (Note 2, Section 2.1) over the six predictable tokens repairs the zero, $\hat{p}(\text{stops} \mid \text{pion}) = (0+1)/(1+6) = 1/7$, since "pion" occurred once and each of the six tokens receives one extra count; but it knows nothing about *why* the sentence is plausible: that a pion, like a muon, is a particle that can stop.

The difficulty grows explosively with n. A vocabulary of $V = 10^4$ words has $V^{n-1} = 10^{16}$ possible contexts for $n = 5$, almost all of which never occur in any corpus. Counting cannot generalise from one context to a similar one: "the muon stops" tells it nothing about "the pion stops".

The neural language model. Bengio, Ducharme, Vincent and Jauvin [1] kept the Markov window (2) but replaced the table of counts by a neural network that *shares* statistical strength between similar contexts. Each token w is mapped to a learned vector $\mathbf{e}(w) \in \mathbb{R}^e$, a row of an *embedding matrix* $E \in \mathbb{R}^{V \times e}$; looking up a row is the same as multiplying the one-hot code of Note 1, Section 3.1, by E^{T}. The $n-1$ context embeddings are concatenated and passed through an MLP (Note 11, Section 4) with a softmax output over the vocabulary:

$$\begin{gathered} \mathbf{x}_t = \big[\mathbf{e}(w_{t-n+1}); \dots; \mathbf{e}(w_{t-1})\big] \in \mathbb{R}^{(n-1)e}, \qquad \mathbf{h}_t = \tanh(U\mathbf{x}_t + \mathbf{b}), \\ p(w_t = k \mid \text{context}) = \operatorname{softmax}(V_{\text{out}}\mathbf{h}_t + \mathbf{b}_y)_k, \end{gathered} \tag{3}$$

with d hidden units, $U \in \mathbb{R}^{d \times (n-1)e}$, $\mathbf{b} \in \mathbb{R}^d$, $V_{\text{out}} \in \mathbb{R}^{V \times d}$ and $\mathbf{b}_y \in \mathbb{R}^V$ (the subscript distinguishes the matrix from the vocabulary size V). The embeddings are trained jointly with the rest by backpropagation. Tokens that occur in similar contexts receive similar gradients and drift to nearby vectors, and then any sentence containing one of them makes the model more confident about sentences with the other. This is the answer to Worked example 1: if $\mathbf{e}(\text{pion}) \approx \mathbf{e}(\text{muon})$, the network assigns "stops" after "pion" roughly the probability it learned after "muon".

Worked example — 2. Where the parameters and the work of a neural language model are

Take $V = 10\,000$ words, embeddings of size $e = 60$, a context of $n - 1 = 4$ words and $d = 100$ hidden units.

- Embedding table: $Ve = 600\,000$ parameters.
- Hidden layer: $d\,((n-1)e + 1) = 100 \times 241 = 24\,100$.
- Output layer: $V(d + 1) = 1\,010\,000$.

Total 1 634 100 parameters — against $V^n = 10^{20}$ entries for a full table of 5-gram probabilities. A forward pass for one prediction costs $240 \times 100 = 24\,000$ multiply–adds in the hidden layer and $100 \times 10\,000 = 10^6$ in the output layer: 98% of the computation is the softmax over the vocabulary. The embedding lookup costs no multiplications at all.

Algorithm 1 Neural n-gram language model: next-token distribution and loss

Require: embedding matrix $E \in \mathbb{R}^{V\times e}$; weights $U \in \mathbb{R}^{d\times(n-1)e}$, $\mathbf{b} \in \mathbb{R}^d$, $V_{\text{out}} \in \mathbb{R}^{V\times d}$, $\mathbf{b}_y \in \mathbb{R}^V$; context $w_{t-n+1}, \dots, w_{t-1}$; target w_t

1: **function** NGRAMLM(context, w_t)
2: $\quad\mathbf{x} \leftarrow$ concatenation of the rows $E_{w_{t-n+1}}, \dots, E_{w_{t-1}}$ ▷ lookup, no arithmetic
3: $\quad\mathbf{h} \leftarrow \tanh(U\mathbf{x} + \mathbf{b})$ ▷ $O(d(n-1)e)$
4: $\quad\mathbf{z} \leftarrow V_{\text{out}}\mathbf{h} + \mathbf{b}_y$; $\mathbf{y} \leftarrow \text{softmax}(\mathbf{z})$ ▷ $O(dV)$: the dominant cost
5: $\quad$**return** $\mathbf{y}$ and the loss $\ell = -\log y_{w_t}$
6: **end function**

Running time — neural n-gram language model

Prediction: $O(d(n-1)e + dV)$ per token: the hidden layer plus the output layer, which dominates for large vocabularies (Worked example 2). *Training*: about three times the forward cost per token (Note 12, Section 3); only the $n - 1$ embedding rows that were looked up receive a gradient, so the update of E is sparse. *Memory*: $Ve + d((n-1)e + 1) + V(d + 1)$ parameters. A hierarchical softmax, which organises the vocabulary as a binary tree and predicts the path to a leaf, reduces the output cost to $O(d \log_2 V)$ [23].

Modern connection — embeddings and tokens in today's language models

Every large language model still starts with the lookup of (3): a table with one learned vector per token. Vocabularies are no longer whole words but subword units built by byte-pair encoding [35], which merges frequent character pairs repeatedly so that rare words are spelled out from common pieces (with byte-level BPE, no input is ever out of vocabulary). Many models also *tie* the input embedding E and the output matrix V_{out}, using the same $V \times e$ matrix for both (this requires $e = d$, or an extra projection), which saves Ve parameters and was found to improve language models [29]. What has changed since 2003 is the context: the fixed window of $n - 1$ words was replaced first by a recurrent state (Section 2) and then by attention over thousands of tokens (Note 15, Section 5).

2 Recurrent neural networks

The neural n-gram model still forgets everything older than $n - 1$ tokens, and its first layer grows with the window. A *recurrent neural network* (RNN) [31, 6] removes the window. It keeps a *hidden state* $\mathbf{h}_t \in \mathbb{R}^d$ and updates it with every new input, using the same weights at every step:

$$\mathbf{a}_t = W\mathbf{h}_{t-1} + U\mathbf{x}_t + \mathbf{b}, \qquad \mathbf{h}_t = \tanh(\mathbf{a}_t), \qquad \mathbf{y}_t = \text{softmax}(\mathbf{z}_t), \quad \mathbf{z}_t = V_{\text{out}}\mathbf{h}_t + \mathbf{b}_y, \tag{4}$$

for $t = 1, \dots, T$, with $W \in \mathbb{R}^{d \times d}$ the *recurrent* weights, $U \in \mathbb{R}^{d \times e}$ the input weights and a fixed initial state $\mathbf{h}_0$ (usually $\mathbf{0}$, or a learned vector). The output layer is chosen for the task, exactly as for an MLP (Note 11, Section 5): a softmax for tokens, a linear unit for a real-valued signal.

A recurrent language model. For text, the input at step t is the embedding of the previous token, $\mathbf{x}_t = \mathbf{e}(w_{t-1})$ with $w_0 =$ <bos>, and $\mathbf{y}_t$ is the predicted distribution of w_t. Since $\mathbf{h}_t$ depends on $\mathbf{h}_{t-1}$, which depends on $\mathbf{h}_{t-2}$, and so on, $\mathbf{h}_t$ is a function of the whole prefix $w_{<t}$: the RNN models (1) without a Markov assumption. The loss of a sequence is the sum of next-token losses,

$$\ell = \sum_{t=1}^{T} \ell_t, \qquad \ell_t = -\log y_{t,\, w_t}, \tag{5}$$

which is the negative log-likelihood of the sequence. Mikolov et al. made such RNN language models practical and showed that they improve on strong n-gram models [22].

Unrolling. Figure 1 draws the recursion in two ways. On the left, a loop: the state feeds back into the cell. On the right, the same network *unrolled* in time: one copy of the cell per step, all copies sharing W, U, V_{out} and the biases. The unrolled graph is an ordinary feed-forward network of depth T whose layers have tied weights, and this is how it is trained (Section 3). Two consequences of sharing are worth stating. The number of parameters,

$$d(d + e + 1) + V(d + 1) \quad \text{(recurrent cell plus output layer)}, \tag{6}$$

does not depend on T, so one network handles sequences of any length; and what the network learns about a pattern at one position applies at every position — weight sharing in time, as a convolution shares weights in space (Note 13, Section 2).

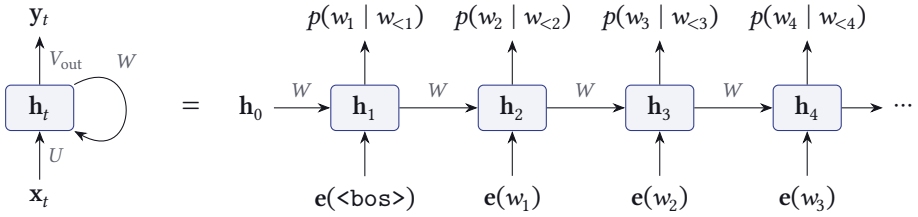


Figure 1. A recurrent network folded (left) and unrolled over four steps as a language model (right). Every copy of the cell uses the same W (state to state), U (input to state) and V_{out} (state to output). The input at step t is the embedding of the previous token, so $\mathbf{h}_t$ summarises $w_{<t}$ and predicts w_t.

Generating text. A trained RNN language model generates a sequence by *ancestral sampling*: start from $\mathbf{h}_0$ and <bos>, compute $\mathbf{y}_1$, draw w_1 from it (Note 2, Section 2.2), feed $\mathbf{e}(w_1)$ back as the next input, and repeat until <eos> is drawn. Each step costs one cell update and one softmax, and the steps are strictly sequential. Graves showed that this simple loop generates convincing text character by character and even handwriting, by predicting pen positions [11]. Section 8 discusses alternatives to plain sampling.

Worked example — 3. A one-unit recurrent network, forward

Take $d = e = 1$, a linear output $y_t = v h_t$ and the squared loss $\ell = \frac{1}{2}\sum_t (y_t - y_t^\star)^2$, so that every number can be followed by hand. Parameters $w = 0.5$ (recurrent), $u = 1$ (input), $b = 0$, $v = 2$; initial state $h_0 = 0$; inputs $x = (1, 0, -1)$ and targets $y^\star = (1, 1, -1)$. Then

t	$a_t = w h_{t-1} + u x_t$	$h_t = \tanh a_t$	$y_t = 2h_t$	$y_t - y_t^\star$	$1 - h_t^2$
1	$0 + 1 = 1$	0.7616	1.5232	0.5232	0.4200
2	$0.3808 + 0 = 0.3808$	0.3634	0.7268	−0.2732	0.8679
3	$0.1817 - 1 = -0.8183$	−0.6741	−1.3483	−0.3483	0.5455

and $\ell = \frac{1}{2}(0.5232^2 + 0.2732^2 + 0.3483^2) = 0.2348$. The input of step 1 is still felt at step 2, where $x_2 = 0$: $h_2 = \tanh(0.5\, h_1)$ is pure memory, halved by w and squashed by the tanh. Section 3 computes the gradients of this example.

In physics — a recurrent network is a discretised dynamical system

Equation (4) is a map $\mathbf{h}_t = F(\mathbf{h}_{t-1}, \mathbf{x}_t)$: a discrete-time dynamical system driven by the input, with the hidden state as its phase-space point. The connection to differential equations is direct. Integrating $\tau\, \dot{\mathbf{h}} = -\mathbf{h} + \tanh(W\mathbf{h} + U\mathbf{x}(t) + \mathbf{b})$, the classic model of a continuous-time neural circuit, with the explicit Euler method and step Δt gives

$$\mathbf{h}_t = \Big(1 - \frac{\Delta t}{\tau}\Big)\mathbf{h}_{t-1} + \frac{\Delta t}{\tau}\tanh(W\mathbf{h}_{t-1} + U\mathbf{x}_t + \mathbf{b})$$

(with $\mathbf{x}_t$ denoting the input sampled at the start of the step), a *leaky* RNN; with $\Delta t = \tau$ it is exactly (4). Going the other way, a residual update $\mathbf{h}_t = \mathbf{h}_{t-1} + \Delta t\, f(\mathbf{h}_{t-1})$ is an Euler step of $\dot{\mathbf{h}} = f(\mathbf{h})$, and neural ordinary differential equations train f directly and integrate it with an adaptive solver [4]. The vocabulary of dynamical systems — fixed points, limit cycles, chaos, Lyapunov exponents — therefore applies to RNNs, and Section 4 uses it to understand their gradients.

2.1 Deep recurrent networks and sequence labelling

An RNN is already deep in time. It can also be made deep at each time step by *stacking* recurrent layers: layer l (written k in Notes 11–13; here k is kept for time gaps, and every width H_k equals d) takes the hidden state of layer $l-1$ at the same step as its input,

$$\mathbf{h}_t^{(l)} = \tanh\big(W^{(l)}\mathbf{h}_{t-1}^{(l)} + U^{(l)}\mathbf{h}_t^{(l-1)} + \mathbf{b}^{(l)}\big), \qquad \mathbf{h}_t^{(0)} = \mathbf{x}_t, \quad l = 1, \dots, L, \tag{7}$$

and the output layer reads the top state $\mathbf{h}_t^{(L)}$. Each layer has its own weights, shared over time. Lower layers then tend to handle local, fast-changing structure and higher layers slower, more abstract structure. With a hidden width d in every layer and input size e, the cells have $d(d+e+1)+(L-1)\,d(2d+1)$ parameters: two layers with $d = 512$ and $e = 256$ give $393\,728 + 524\,800 = 918\,528$. In practice two to four layers are common for gated cells (Section 5), sometimes with residual connections between layers (Note 12, Section 10.3) when the stack is deeper.

Beyond language models. Nothing in (4) or (7) is specific to text. When every input position has its own label — *sequence labelling* — the output at step t is the label of position t: the part of speech of each word, whether each word belongs to a named entity, the character drawn by each stretch of pen trajectory in on-line handwriting recognition [10], or the phoneme spoken in each frame of

audio. The loss is again a sum over time of per-step losses (5), with w_t replaced by the label $y_t^\star$. A left-to-right network labels position t using only positions $1, \dots, t$; Section 9 removes this restriction when the whole input is available.

Algorithm 2 Forward pass of a deep RNN over one sequence (language model or sequence labelling)

Require: inputs $\mathbf{x}_1, \dots, \mathbf{x}_T$ (embeddings of $w_0, \dots, w_{T-1}$ for a language model); targets $y_1^\star, \dots, y_T^\star$; layers $l = 1, \dots, L$ with $W^{(l)}, U^{(l)}, \mathbf{b}^{(l)}$; output $V_{\text{out}}, \mathbf{b}_y$

function RNNForward($\mathbf{x}_{1:T}$, $y^\star_{1:T}$)
  $\mathbf{h}_0^{(l)} \leftarrow \mathbf{0}$ for every layer; $\ell \leftarrow 0$
  for $t = 1, \dots, T$ **do** ▷ sequential in t
    $\mathbf{h}_t^{(0)} \leftarrow \mathbf{x}_t$
    for $l = 1, \dots, L$ **do**
      $\mathbf{a}_t^{(l)} \leftarrow W^{(l)}\mathbf{h}_{t-1}^{(l)} + U^{(l)}\mathbf{h}_t^{(l-1)} + \mathbf{b}^{(l)}$; $\mathbf{h}_t^{(l)} \leftarrow \tanh(\mathbf{a}_t^{(l)})$ ▷ $O(d^2 + de)$
    end for
    $\mathbf{y}_t \leftarrow \text{softmax}(V_{\text{out}}\mathbf{h}_t^{(L)} + \mathbf{b}_y)$; $\ell \leftarrow \ell - \log y_{t,\,y_t^\star}$ ▷ $O(dV)$
  end for
  return ℓ, and all $\mathbf{h}_t^{(l)}$, $\mathbf{y}_t$ for the backward pass
end function

Running time — RNN forward pass

Time: $O\big(T(Ld^2 + de + dV)\big)$ per sequence of length T (input size e in the first layer, d above it): one matrix–vector product per layer and step, plus the output layer. A mini-batch of n' sequences turns the products into matrix–matrix products, $O(n'T(Ld^2 + dV))$, but the loop over t cannot be parallelised: step t needs $\mathbf{h}_{t-1}$. *Memory*: $d(d+e+1) + (L-1)d(2d+1) + V(d+1)$ parameters (Section 2.1), plus the $O(TLd)$ stored states needed by the backward pass. *Generation*: the same cost per produced token, and only the current state ($O(Ld)$ numbers) must be kept between tokens — an advantage over attention with its key–value cache (Note 15, Section 6.2) that returns in Section 11.

In physics — autoregressive networks as wave functions and Boltzmann samplers

The chain rule (1) is equally useful for physical configurations. Order the N spins of a lattice along a path and model $p(s_1, \dots, s_N) = \prod_i p(s_i \mid s_{<i})$ with an RNN that reads the spins one by one. Two properties make this attractive. The probability of a configuration is computed exactly, normalisation included, and independent configurations are drawn exactly by ancestral sampling — no Markov chain, no autocorrelation time, no burn-in. Hibat-Allah et al. used such recurrent networks as variational wave functions for quantum spin models, with $|\psi(\mathbf{s})|^2$ given by the autoregressive product [16]. The price is the sequential order imposed on a lattice that has none: neighbours in two dimensions can be far apart along the path, and the hidden state must carry that information.

3 Backpropagation through time

The unrolled network of Figure 1 is a feed-forward computational graph, so its gradient is obtained by backpropagation (Note 12, Section 3) applied to that graph. Run on an unrolled recurrent network, the algorithm is called *backpropagation through time* (BPTT) [40, 31]. Only one feature is new: every parameter is used at every step, and its gradient is the sum of the contributions of all steps.

Derivation. Write the loss as $\ell = \sum_t \ell_t$ (5) and take one layer, as in (4). The gradient of a softmax with cross-entropy with respect to the logits is $\partial \ell_t / \partial \mathbf{z}_t = \mathbf{y}_t - \mathbf{t}_t$, with $\mathbf{t}_t$ the one-hot code of the target (Note 12, Section 2). The state $\mathbf{h}_t$ influences the loss along two paths: through its own output $\mathbf{z}_t$, and through the next state $\mathbf{h}_{t+1}$, which carries it into all later losses. Define

$$\bar{\mathbf{h}}_t = \frac{\partial \ell}{\partial \mathbf{h}_t}, \qquad \boldsymbol{\delta}_t = \frac{\partial \ell}{\partial \mathbf{a}_t}.$$

The chain rule over the two paths, with all gradients written as column vectors, reads

$$\bar{\mathbf{h}}_t = \Big(\frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_t}\Big)^{\mathsf{T}} \frac{\partial \ell}{\partial \mathbf{z}_t} + \Big(\frac{\partial \mathbf{a}_{t+1}}{\partial \mathbf{h}_t}\Big)^{\mathsf{T}} \frac{\partial \ell}{\partial \mathbf{a}_{t+1}}, \qquad \boldsymbol{\delta}_t = \Big(\frac{\partial \mathbf{h}_t}{\partial \mathbf{a}_t}\Big)^{\mathsf{T}} \bar{\mathbf{h}}_t.$$

Only ℓ_t depends on $\mathbf{z}_t$, so $\partial \ell / \partial \mathbf{z}_t = \mathbf{y}_t - \mathbf{t}_t$. The Jacobians are read off (4): $\partial \mathbf{z}_t / \partial \mathbf{h}_t = V_{\text{out}}$, $\partial \mathbf{a}_{t+1} / \partial \mathbf{h}_t = W$ (since $\mathbf{a}_{t+1} = W\mathbf{h}_t + U\mathbf{x}_{t+1} + \mathbf{b}$), and $\partial \mathbf{h}_t / \partial \mathbf{a}_t = \operatorname{diag}(1 - \mathbf{h}_t^2)$, which is symmetric. Substituting,

$$\bar{\mathbf{h}}_t = V_{\text{out}}^{\mathsf{T}}(\mathbf{y}_t - \mathbf{t}_t) + W^{\mathsf{T}} \boldsymbol{\delta}_{t+1}, \qquad \boldsymbol{\delta}_t = (1 - \mathbf{h}_t^2) \odot \bar{\mathbf{h}}_t, \tag{8}$$

with $\boldsymbol{\delta}_{T+1} = \mathbf{0}$ and $\tanh'(a) = 1 - \tanh^2(a)$ applied element-wise. The recursion runs backwards in time, from $t = T$ to $t = 1$. For the weights, $a_{t,i} = \sum_j W_{ij} h_{t-1,j} + \dots$ gives $\partial a_{t,i} / \partial W_{ij} = h_{t-1,j}$, so the copy of W used at step t receives $\delta_{t,i} h_{t-1,j}$, the outer product $\boldsymbol{\delta}_t \mathbf{h}_{t-1}^{\mathsf{T}}$. The shared W sums these over t: each shared parameter collects one term per step, exactly as a shared convolution weight collects one term per position (Note 13, Section 5.1):

$$\frac{\partial \ell}{\partial W} = \sum_{t=1}^{T} \boldsymbol{\delta}_t \, \mathbf{h}_{t-1}^{\mathsf{T}}, \quad \frac{\partial \ell}{\partial U} = \sum_{t=1}^{T} \boldsymbol{\delta}_t \, \mathbf{x}_t^{\mathsf{T}}, \quad \frac{\partial \ell}{\partial \mathbf{b}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t, \quad \frac{\partial \ell}{\partial V_{\text{out}}} = \sum_{t=1}^{T} (\mathbf{y}_t - \mathbf{t}_t) \, \mathbf{h}_t^{\mathsf{T}}, \tag{9}$$

and $\partial \ell / \partial \mathbf{b}_y = \sum_t (\mathbf{y}_t - \mathbf{t}_t)$. For a language model the input $\mathbf{x}_t$ is the row of E for token w_{t-1}, and that row receives $U^{\mathsf{T}} \boldsymbol{\delta}_t$. The recursion (8) is that of Note 12, Section 3.2, with one difference: the matrix W is the same at every "layer". Section 4 shows that this is what makes long sequences hard.

Worked example — 4. Backpropagation through time by hand

Continue Worked example 3. With a linear output and squared loss, $\partial \ell_t / \partial y_t = y_t - y_t^\star$ and the first term of (8) becomes $v(y_t - y_t^\star)$; the recursion reads $\bar{h}_t = v(y_t - y_t^\star) + w\,\delta_{t+1}$, $\delta_t = (1 - h_t^2)\,\bar{h}_t$. Going backwards:

$$\begin{aligned}
t = 3: \quad & \bar{h}_3 = 2(-0.3483) = -0.6966, && \delta_3 = 0.5455 \times (-0.6966) = -0.3800, \\
t = 2: \quad & \bar{h}_2 = 2(-0.2732) + 0.5(-0.3800) = -0.7364, && \delta_2 = 0.8679 \times (-0.7364) = -0.6392, \\
t = 1: \quad & \bar{h}_1 = 2(0.5232) + 0.5(-0.6392) = 0.7268, && \delta_1 = 0.4200 \times 0.7268 = 0.3052.
\end{aligned}$$

The parameter gradients (9) sum one term per step:

$$\begin{aligned}
\partial \ell / \partial w &= \delta_1 h_0 + \delta_2 h_1 + \delta_3 h_2 = 0 - 0.4868 - 0.1381 = -0.6249, \\
\partial \ell / \partial u &= \delta_1 x_1 + \delta_2 x_2 + \delta_3 x_3 = 0.3052 + 0 + 0.3800 = 0.6852, \\
\partial \ell / \partial b &= \delta_1 + \delta_2 + \delta_3 = -0.714, \\
\partial \ell / \partial v &= \textstyle\sum_t (y_t - y_t^\star) h_t = 0.3985 - 0.0993 + 0.2348 = 0.534.
\end{aligned}$$

Central differences with step 10^{-6} (Note 12, Section 4) reproduce all four values. Note how the error at $t = 3$ reaches h_1: through the factor $\partial h_3 / \partial h_1 = (1 - h_3^2)\, w\, (1 - h_2^2)\, w = 0.118$. Each step back in time multiplies by $w(1 - h^2) \le 0.5$ here, so the influence of distant steps on the gradient fades geometrically — the theme of Section 4.

Algorithm 3 Backpropagation through time for one sequence (one tanh layer, softmax output)

Require: stored $\mathbf{x}_t$, $\mathbf{h}_t$ ($t = 0, \dots, T$) and $\mathbf{y}_t$ from Algorithm 2; one-hot targets $\mathbf{t}_t$

function BPTT
  set $\nabla W, \nabla U, \nabla \mathbf{b}, \nabla V_{\text{out}}, \nabla \mathbf{b}_y$ to zero; $\boldsymbol{\delta}_{T+1} \leftarrow \mathbf{0}$
  for $t = T, T-1, \dots, 1$ **do** ▷ backwards in time
    $\bar{\mathbf{z}}_t \leftarrow \mathbf{y}_t - \mathbf{t}_t$; $\nabla V_{\text{out}} \mathrel{+}= \bar{\mathbf{z}}_t \mathbf{h}_t^{\mathsf{T}}$; $\nabla \mathbf{b}_y \mathrel{+}= \bar{\mathbf{z}}_t$ ▷ $O(dV)$
    $\bar{\mathbf{h}}_t \leftarrow V_{\text{out}}^{\mathsf{T}} \bar{\mathbf{z}}_t + W^{\mathsf{T}} \boldsymbol{\delta}_{t+1}$ ▷ two paths: output and future
    $\boldsymbol{\delta}_t \leftarrow (1 - \mathbf{h}_t^2) \odot \bar{\mathbf{h}}_t$
    $\nabla W \mathrel{+}= \boldsymbol{\delta}_t \mathbf{h}_{t-1}^{\mathsf{T}}$; $\nabla U \mathrel{+}= \boldsymbol{\delta}_t \mathbf{x}_t^{\mathsf{T}}$; $\nabla \mathbf{b} \mathrel{+}= \boldsymbol{\delta}_t$ ▷ $O(d^2 + de)$
    (language model: add $U^{\mathsf{T}} \boldsymbol{\delta}_t$ to the gradient of the embedding row of w_{t-1})
  end for
  return all gradients
end function

Running time — backpropagation through time

Time: $O(T(d^2 + de + dV))$ per sequence, about twice the forward pass, so a training step costs about three forward passes, as for any network (Note 12, Section 3). *Memory*: all T states $\mathbf{h}_t$ (and the outputs, or the logits from which they are recomputed) must be stored, $O(Td)$ per layer and sequence, or $O(n'TLd)$ for a mini-batch — the memory grows linearly with the length of the sequence. Libraries obtain the same algorithm automatically by reverse-mode automatic differentiation of the unrolled loop (Note 12, Section 3.5). Activation checkpointing (Note 12, modern box of Section 3.5), with time steps in the role of layers, reduces the memory to $O(\sqrt{T}d)$ for one extra forward pass.

3.1 Truncated backpropagation through time

For a very long sequence — a book read as one stream, a year of sensor data, an on-line stream that never ends — the memory of full BPTT is prohibitive and one update per sequence is far too rare. *Truncated* BPTT cuts the sequence into consecutive chunks of k steps. It runs the forward pass over one chunk, starting from the final state of the previous chunk, backpropagates only within the chunk, updates the parameters, and moves on. The state is carried forward across chunk boundaries, so the forward pass still sees the whole past; the gradient is truncated, because no error is propagated across a boundary. More generally one runs k_1 new forward steps between updates and backpropagates over the last $k_2 \geq k_1$ steps; $k_1 = k_2 = k$ is the common choice. A dependency longer than k steps can still be *used* by the network, but is not *learned* through the gradient — unless it is learned at shorter range and generalises.

Algorithm 4 Truncated BPTT with gradient clipping over one long sequence ($k_1 = k_2 = k$)

Require: a long sequence $\mathbf{x}_1, \dots, \mathbf{x}_T$ with targets; chunk length k; learning rate η (or an optimiser, Note 7, Section 6); clipping threshold G

function TruncatedBPTT($\mathbf{x}_{1:T}$)
  $\mathbf{h} \leftarrow \mathbf{0}$
  for $s = 0, k, 2k, \dots$ while $s < T$ **do**
    run Algorithm 2 on $\mathbf{x}_{s+1}, \dots, \mathbf{x}_{\min(s+k,\,T)}$ starting from the state $\mathbf{h}$
    run Algorithm 3 on this chunk only ▷ the incoming $\mathbf{h}$ is a constant
    $\mathbf{g} \leftarrow$ all parameter gradients as one vector; **if** $\|\mathbf{g}\| > G$ **then** $\mathbf{g} \leftarrow G\,\mathbf{g}/\|\mathbf{g}\|$ ▷ Section 4.1
    $\theta \leftarrow \theta - \eta\,\mathbf{g}$; $\mathbf{h} \leftarrow$ final state of the chunk ▷ carried, not differentiated
  end for
end function

Running time — truncated BPTT

Time: the same $O(T(d^2 + de + dV))$ per pass over the sequence as full BPTT, now spread over $\lceil T/k \rceil$ updates instead of one. With $k_2 > k_1$ the backward work grows by the factor k_2/k_1. *Memory*: $O(kd)$ stored states per layer, independent of T. For example, a stream of $T = 10^6$ steps through an LSTM with $d = 1024$ (which stores about six vectors of size d per step) would need $6 \times 1024 \times 10^6 \times 4$ bytes ≈ 24.6 GB of activations for full BPTT in 32-bit floats, and 2.5 MB for chunks of $k = 100$ steps. The price is the limited range of dependencies that the gradient can see.

4 Exploding and vanishing gradients through time

How does the loss at time T depend on the state at an earlier time t? Differentiating the update $\mathbf{h}_s = \tanh(W\mathbf{h}_{s-1} + U\mathbf{x}_s + \mathbf{b})$ gives the one-step Jacobian

$$\frac{\partial \mathbf{h}_s}{\partial \mathbf{h}_{s-1}} = D_s W, \qquad D_s = \operatorname{diag}(1 - \mathbf{h}_s^2),$$

and the chain rule multiplies these Jacobians along the path from t to T:

$$\frac{\partial \ell_T}{\partial \mathbf{h}_t} = \left(\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_t}\right)^{\mathsf{T}} \frac{\partial \ell_T}{\partial \mathbf{h}_T}, \qquad \frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_t} = D_T W \, D_{T-1} W \cdots D_{t+1} W = \prod_{s=t+1}^{T} D_s W \tag{10}$$

(the product ordered with later times on the left). Unrolling (8) gives the same product: the contribution of ℓ_T to $\bar{\mathbf{h}}_t$ is $W^{\mathsf{T}} D_{t+1} \cdots W^{\mathsf{T}} D_T \, \partial \ell_T / \partial \mathbf{h}_T$. The gradient of W in (9) sums such terms over all pairs $t < T$, so the question is how fast these products grow or shrink with the gap $k = T - t$.

Derivation: the bound by spectral norms. Let $\|A\|$ be the spectral norm, the largest singular value, so that $\|A\mathbf{u}\| \le \|A\| \, \|\mathbf{u}\|$ and hence $\|AB\| \le \|A\| \, \|B\|$. The diagonal matrix D_s holds the derivatives of the activation, $D_s = \operatorname{diag}(g'(\mathbf{a}_s))$, so its norm is $\max_j |g'(a_{s,j})| \le \gamma$, with $\gamma = 1$ for tanh ($g' = 1 - h^2$) and $\gamma = \frac{1}{4}$ for the logistic sigmoid ($g' = h(1-h)$). Applying the product inequality k times,

$$\left\| \frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_t} \right\| \le \prod_{s=t+1}^{T} \|D_s\| \, \|W\| \le \left(\gamma \, \|W\|\right)^k, \qquad k = T - t. \tag{11}$$

If $\gamma \, \|W\| < 1$, the contribution of a step k steps back shrinks at least geometrically: a *sufficient* condition for *vanishing gradients*. Conversely, the gradient can only *explode* if $\gamma \, \|W\| > 1$: a *necessary* condition [2, 26]. The same analysis for depth instead of time is in Note 12, Section 9.2. The time version is worse in one respect: there, each layer has its own matrix, and the factors need not conspire; here the *same* matrix W is multiplied in k times, and sequences of hundreds or thousands of steps are common.

Eigenvalues decide the long run. The bound (11) uses the norm, which can be larger than the eigenvalues. In the linear case ($D_s = I$) the product is the matrix power W^k. If $W = Q\Lambda Q^{-1}$ is diagonalisable, $W^k = Q\Lambda^k Q^{-1}$: the component along eigenvector j is multiplied by λ_j^k. Components with $|\lambda_j| < 1$ die out and those with $|\lambda_j| > 1$ blow up, and for large k the growth rate is set by the *spectral radius* $\rho(W) = \max_j |\lambda_j|$; in general $\left\|W^k\right\|^{1/k} \to \rho(W)$ (Gelfand's formula). The norm controls what can happen in a few steps, the spectral radius what happens over many.

Worked example — 5. Scalar recurrences, and a matrix whose norm misleads

Linear. For $h_t = wh_{t-1}$, $\partial h_T/\partial h_{T-k} = w^k$. With $w = 0.9$: 0.349, 0.122 and 0.0052 for $k = 10, 20, 50$. With $w = 1.1$: 2.59, 6.73 and 117. Moving the weight from ten per cent below one to ten per cent above it separates a gradient that is gone after fifty steps from one that is a hundred times too large.

With tanh. For $h_t = \tanh(wh_{t-1})$ and $w = 1.1$, start at $h_0 = 0.1$. The state first grows (the slope near zero is 1.1) and then settles at the fixed point $h^* = \tanh(1.1h^*) = 0.503$, where the derivative of the map is $w(1 - h^{*2}) = 0.822 < 1$. After fifty steps $h_{50} = 0.503$ and $\partial h_{50}/\partial h_0 = \prod_t w(1 - h_t^2) = 0.0079$: the saturation of the tanh has turned $|w| > 1$ into a *vanishing* gradient. With $w = 2$, $h^* = 0.958$, the slope at the fixed point is 0.166, and $\partial h_{50}/\partial h_0 = 5 \times 10^{-36}$. An undriven tanh unit started away from zero cannot keep exploding: it settles at a stable fixed point, where the slope $w(1 - h^{*2})$ is below one. Sustained explosion needs units kept in the near-linear regime around $h = 0$ (at $h = 0$ exactly, $\partial h_k/\partial h_0 = w^k$), for example by the input, or several interacting units whose dynamics are chaotic (Figure 2).

Non-normal matrix. The matrix $A = \begin{pmatrix} 0.5 & 2 \\ 0 & 0.5 \end{pmatrix}$ has both eigenvalues equal to 0.5, so $\rho(A) = 0.5$, but $\|A\| = 2.12$. Its powers are $A^k = \begin{pmatrix} 0.5^k & 4k\,0.5^k \\ 0 & 0.5^k \end{pmatrix}$, with norms 2.03, 1.51 and 0.039 for $k = 2, 3, 10$: a transient of two or three steps during which the norm stays near two, then the decay at rate 0.5 predicted by the spectral radius. The bound $\|A\|^{10} = 1817$ is useless here.

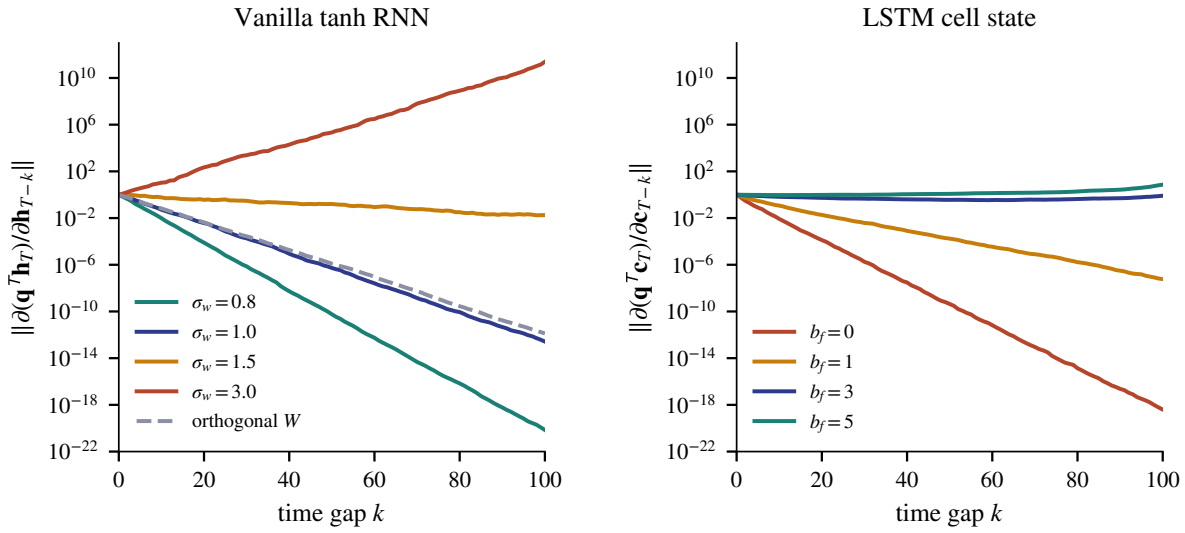


Figure 2. Norm of the gradient that a unit probe vector $\mathbf{q}$ applied to the state at time T sends back k steps, at initialisation (median of 20 random networks, $d = 64$, inputs ± 1 at random). Left: tanh RNNs with Gaussian recurrent weights $W_{ij} \sim \mathcal{N}(0, \sigma_w^2/d)$, whose spectral radius is close to σ_w, and with an orthogonal W; the state is $\mathbf{h}$. At $k = 100$ the norm is 7×10^{-21} for $\sigma_w = 0.8$, 3×10^{-13} for $\sigma_w = 1$, 0.018 for $\sigma_w = 1.5$ and 2×10^{11} for $\sigma_w = 3$; orthogonal weights give 1×10^{-12}. Right: LSTMs (Section 5) with recurrent weights $\mathcal{N}(0, 1/d)$ for every gate ($\sigma_w = 1$), input weights $\mathcal{N}(0, 1)$ and forget-gate bias b_f; the state is the cell $\mathbf{c}$, and all paths are included. At $k = 100$: 4×10^{-19}, 6×10^{-8}, 0.8 and 7 for $b_f = 0, 1, 3, 5$. Colours are not matched between the panels.

Figure 2 (left) measures these products in random tanh networks whose recurrent weights are drawn as $W_{ij} \sim \mathcal{N}(0, \sigma_w^2/d)$; the *gain* σ_w (the σ_w of the physics box of Note 12, Section 7) then sets the spectral radius, $\rho(W) \approx \sigma_w$. Each curve is close to a straight line on the logarithmic axis: the gradient changes by a roughly constant factor per step, as (11) suggests. For small gains it vanishes; for $\sigma_w = 3$ it explodes by eleven orders of magnitude over a hundred steps. Even at $\sigma_w = 1$ it vanishes, because

the inputs and the state keep units away from zero, where the derivatives in D_s are below one, so every factor shrinks a little; the gain must be raised to about 1.5 before the decay becomes slow, and there the network is close to the boundary between order and chaos (box below). An orthogonal W, whose singular values are all one, still vanishes, because the factors D_s have entries below one whenever units are away from zero.

Consequences. Exploding gradients are spectacular: the loss jumps, parameters fly off, and training may produce overflow. Vanishing gradients are silent but worse. The gradient of W in (9) is a sum of terms from all gaps k, and the terms with large k are exponentially smaller than those with small k; the gradient is dominated by short-range dependencies, and a dependency spanning fifty steps — the subject of a sentence that decides the form of a verb many words later, or a slow drift in a sensor — produces a signal buried under the short-range ones. Bengio, Simard and Frasconi showed that this is a structural dilemma: a recurrent network that stores information robustly, against small perturbations of its state, operates in a regime where the gradient with respect to the past vanishes [2].

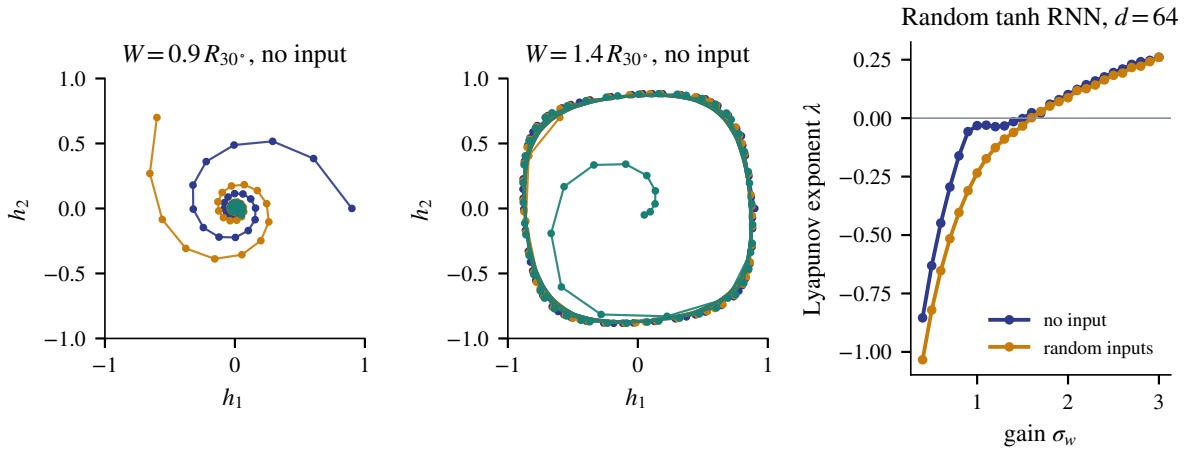


Figure 3. A recurrent network as a dynamical system. Left and centre: trajectories of a two-unit network $\mathbf{h}_t = \tanh(W\mathbf{h}_{t-1})$ with W a rotation by 30° scaled by 0.9 or 1.4, from three starting points. With 0.9 every trajectory spirals into the stable fixed point $\mathbf{0}$; with 1.4 the origin is unstable and all trajectories settle on the same attracting closed curve, the discrete-time counterpart of a *limit cycle*. Right: largest Lyapunov exponent λ of random tanh networks ($d = 64$, mean of five networks) against the gain σ_w, without input and with the random inputs of Figure 2. With inputs, $\lambda = -0.40$ at $\sigma_w = 0.8$, -0.23 at $\sigma_w = 1$ and $+0.26$ at $\sigma_w = 3$, comparable to the average per-step slopes of $\ln\|\cdot\|$ in Figure 2 (-0.47, -0.29 and $+0.26$; on the base-10 axis of that figure, divide by $\ln 10 \approx 2.30$). Both curves change sign near $\sigma_w \approx 1.6$.

In physics — Lyapunov exponents: vanishing and exploding gradients in time

For a dynamical system $\mathbf{h}_t = F(\mathbf{h}_{t-1})$, a small perturbation $\boldsymbol{\epsilon}_0$ of the initial state evolves as $\boldsymbol{\epsilon}_k \approx \left(\prod_s J_s\right)\boldsymbol{\epsilon}_0$, with J_s the Jacobian along the trajectory — the same product as (10). Its typical growth rate, $\lambda = \lim_{k\to\infty} \frac{1}{k} \ln\|\boldsymbol{\epsilon}_k\|$, is the largest *Lyapunov exponent*. Numerically, one propagates a tangent vector, $\boldsymbol{\epsilon} \leftarrow J_s\boldsymbol{\epsilon}$, renormalises it to unit length after each step, and averages the logarithms of the growth factors. A negative exponent means that the system forgets its initial condition and settles onto a stable fixed point (Figure 3, left, where $\lambda = \ln 0.9 = -0.105$); on an attracting closed curve (centre) the largest exponent is zero, because displacements along the curve neither grow nor shrink; a positive exponent means chaos, with nearby trajectories separating as $e^{\lambda k}$, and $1/\lambda$ is then the *Lyapunov time*, over which a small error grows by a factor e. Backpropagation applies the transposed Jacobians to a gradient, and transposition

does not change the asymptotic growth rate, so *vanishing gradients are a negative Lyapunov exponent and exploding gradients a positive one*: $\|\partial \ell_T / \partial \mathbf{h}_{T-k}\| \sim e^{\lambda k}$. Figure 3 (right) measures λ for random tanh networks; the measured exponents are comparable to the natural-log slopes of Figure 2. For large continuous-time random networks Sompolinsky, Crisanti and Sommers found a sharp transition to chaos at gain $\sigma_w = 1$ [38], and a driving input suppresses chaos [30]: in the right panel the input lowers λ strongly below the transition (-0.235 against -0.032 at $\sigma_w = 1$). In these small networks ($d = 64$) both curves cross zero only near $\sigma_w \approx 1.6$, and the input-free curve stays close to zero between $\sigma_w = 1$ and 1.5. The depth version of this order–chaos transition is the physics box of Note 12, Section 7; initialising near $\lambda = 0$ is the recurrent counterpart of its critical initialisation.

4.1 First remedies: clipping and orthogonal initialisation

Gradient clipping. Explosion is handled by gradient clipping (Note 7, Section 7; Algorithm 4) [26], standard in most recurrent and transformer training recipes. The picture behind it is a loss surface with steep "cliffs" — regions where a small change of W crosses a *bifurcation*, a qualitative change of the dynamics such as a fixed point turning unstable (Figure 3, left versus centre) — on which an unclipped step would throw the parameters far away. It does nothing for vanishing gradients.

Orthogonal initialisation. An orthogonal matrix, $Q^{\mathsf{T}}Q = QQ^{\mathsf{T}} = I$, preserves norms: $\|Q\mathbf{u}\|^2 = \mathbf{u}^{\mathsf{T}}Q^{\mathsf{T}}Q\mathbf{u} = \|\mathbf{u}\|^2$, and so does its transpose. All its eigenvalues have modulus one and all its singular values equal one. Initialising W as a random orthogonal matrix therefore makes the linear part of every factor in (10) norm-preserving, so that at initialisation the products neither grow nor shrink because of W; deviations come only from the derivatives D_s, which are close to I while the units are near zero [33]. For example, the rotation by 60° maps $(3, 4)$ to $(-1.964, 4.598)$, again of norm 5, and its powers never change the norm. A related recipe for ReLU recurrent networks (IRNN) initialises $W = I$ and the biases to zero, so that the state is copied unchanged until learning modifies it [21]. Both are good starts, but neither survives training automatically, and Figure 2 shows that with saturating units an orthogonal W still lets the gradient vanish. The durable fix is architectural.

Algorithm 5 Random orthogonal initialisation of a recurrent matrix

Require: size d; gain σ_w (usually 1)

function OrthogonalInit(d, σ_w)
  draw $A \in \mathbb{R}^{d \times d}$ with independent $\mathcal{N}(0, 1)$ entries
  $Q, R \leftarrow \mathrm{QR}(A)$ ▷ Gram–Schmidt or Householder, $O(d^3)$
  $Q \leftarrow Q\ \mathrm{diag}(\mathrm{sign}(R_{11}), \dots, \mathrm{sign}(R_{dd}))$ ▷ makes Q uniformly distributed
  return $W = \sigma_w Q$
end function

Running time — orthogonal initialisation

Time: $O(d^3)$ for the QR factorisation, once before training; $d = 1024$ takes about 10^9 operations, a fraction of a second. *Memory*: $O(d^2)$, the size of W itself. No cost during training or prediction. Without the sign correction the distribution of Q depends on the convention of the QR routine; with it, Q is uniformly (Haar) distributed over the orthogonal group.

5 Long short-term memory

The *long short-term memory* (LSTM) of Hochreiter and Schmidhuber [17], in the form with a forget gate introduced by Gers, Schmidhuber and Cummins [7], adds to the hidden state $\mathbf{h}_t$ a second state, the *cell* $\mathbf{c}_t \in \mathbb{R}^d$, which is updated *additively* and read and written through multiplicative *gates*. Each gate is a vector of numbers between 0 and 1 computed from the current input and the previous hidden state:

$$\begin{aligned} \mathbf{i}_t &= \sigma\big(W_i\mathbf{h}_{t-1} + U_i\mathbf{x}_t + \mathbf{b}_i\big) && \text{input gate: how much new content to write,} \\ \mathbf{f}_t &= \sigma\big(W_f\mathbf{h}_{t-1} + U_f\mathbf{x}_t + \mathbf{b}_f\big) && \text{forget gate: how much of the old cell to keep,} \\ \mathbf{o}_t &= \sigma\big(W_o\mathbf{h}_{t-1} + U_o\mathbf{x}_t + \mathbf{b}_o\big) && \text{output gate: how much of the cell to expose,} \end{aligned} \tag{12}$$

with the logistic sigmoid σ (Note 11, Section 2). The cell and the hidden state are then updated as

$$\tilde{\mathbf{c}}_t = \tanh\big(W_c\mathbf{h}_{t-1} + U_c\mathbf{x}_t + \mathbf{b}_c\big), \qquad \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \tag{13}$$

The candidate $\tilde{\mathbf{c}}_t$ is what a plain RNN would compute. The cell keeps a gated fraction of its old content and adds a gated fraction of the candidate; the hidden state, which feeds the output and the gates of the next step, is a gated, squashed view of the cell. The gates are soft switches: with $\mathbf{f}_t = \mathbf{1}$, $\mathbf{i}_t = \mathbf{0}$ the cell is copied unchanged, with $\mathbf{f}_t = \mathbf{0}$ it is erased, and every value in between is possible and differentiable.

Worked example — 6. One LSTM step by hand

Take one unit ($d = e = 1$), previous state $h_{t-1} = 0.5$, previous cell $c_{t-1} = 1$, input $x_t = 1$, all biases zero and the weights $(W_i, U_i) = (-2, 1)$, $(W_f, U_f) = (2, 1)$, $(W_o, U_o) = (2, -1)$, $(W_c, U_c) = (0, 1)$. The pre-activations are

$$a_i = -2(0.5) + 1 = 0, \quad a_f = 2(0.5) + 1 = 2, \quad a_o = 2(0.5) - 1 = 0, \quad a_c = 0 + 1 = 1,$$

so $i_t = \sigma(0) = 0.5$, $f_t = \sigma(2) = 0.881$, $o_t = 0.5$ and $\tilde{c}_t = \tanh 1 = 0.762$. Then

$$c_t = 0.881\times1 + 0.5\times0.762 = 0.881 + 0.381 = 1.262, \qquad h_t = 0.5\tanh(1.262) = 0.5\times0.852 = 0.426.$$

The cell has grown beyond 1, which a tanh unit could never do: it is not squashed, only the read-out is. Along the direct path, $\partial c_t/\partial c_{t-1} = f_t = 0.881$ — no weight and no tanh derivative appear in this factor.

Derivation: why the additive cell helps. Differentiate (13) with respect to the previous cell. The cell enters $\mathbf{c}_t$ directly through $\mathbf{f}_t \odot \mathbf{c}_{t-1}$, and indirectly through $\mathbf{h}_{t-1} = \mathbf{o}_{t-1} \odot \tanh(\mathbf{c}_{t-1})$, on which all gates and the candidate of step t depend:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \operatorname{diag}(\mathbf{f}_t) + \sum_{q\in\{i,f,c\}} \frac{\partial \mathbf{c}_t}{\partial \mathbf{a}_t^{(q)}} W_q \operatorname{diag}\big(\mathbf{o}_{t-1} \odot (1 - \tanh^2 \mathbf{c}_{t-1})\big), \tag{14}$$

where $\mathbf{a}_t^{(q)}$ is the pre-activation of gate q (or of the candidate). These factors are diagonal: with $\sigma' = \sigma(1-\sigma)$ (Note 11, Section 2),

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{a}_t^{(i)}} = \operatorname{diag}\big(\tilde{\mathbf{c}}_t \odot \mathbf{i}_t \odot (1 - \mathbf{i}_t)\big), \quad \frac{\partial \mathbf{c}_t}{\partial \mathbf{a}_t^{(f)}} = \operatorname{diag}\big(\mathbf{c}_{t-1} \odot \mathbf{f}_t \odot (1 - \mathbf{f}_t)\big), \quad \frac{\partial \mathbf{c}_t}{\partial \mathbf{a}_t^{(c)}} = \operatorname{diag}\big(\mathbf{i}_t \odot (1 - \tilde{\mathbf{c}}_t^2)\big).$$

The first term of (14) is the new element. Keeping only it along a path of k steps,

$$\left.\frac{\partial \mathbf{c}_T}{\partial \mathbf{c}_{T-k}}\right|_{\text{direct}} = \operatorname{diag}\big(\mathbf{f}_T \odot \mathbf{f}_{T-1} \odot \cdots \odot \mathbf{f}_{T-k+1}\big) \;:$$

no recurrent matrix and no derivative of a squashing function, only the forget gates. If the network keeps its forget gates near one for a unit, error flows back along that unit's cell for as long as the gate stays open — Hochreiter and Schmidhuber's "constant error carousel". Unrolling the update shows the same thing in the forward direction:

$$\mathbf{c}_t = \sum_{t' \le t} \left(\mathbf{f}_t \odot \cdots \odot \mathbf{f}_{t'+1} \right) \odot \mathbf{i}_{t'} \odot \tilde{\mathbf{c}}_{t'} \qquad (\text{with } \mathbf{c}_0 = \mathbf{0}),$$

a sum of everything ever written, each entry discounted by the forget gates passed since. The network *learns*, through the gates, when to remember and when to forget, and the gradient no longer has to fight a fixed contraction factor. The indirect terms of (14) still contain W and can still vanish or explode, which is why clipping remains useful.

Forget-gate bias. At initialisation the gate pre-activations are near zero, so $\mathbf{f}_t \approx \sigma(\mathbf{b}_f)$. With $\mathbf{b}_f = \mathbf{0}$ every forget gate is about $\frac{1}{2}$ and the direct path decays like 2^{-k}, no better than a poorly scaled plain RNN. Initialising $\mathbf{b}_f$ to 1 or more makes the network remember by default and learn to forget, rather than the reverse; Jozefowicz, Zaremba and Sutskever found that a bias of 1 markedly improves LSTMs on several tasks [20]. With the gate held at $\sigma(b_f)$ the direct path after $k = 20$ steps is

$$\sigma(b_f)^{20} = 9.5 \times 10^{-7}, \quad 1.9 \times 10^{-3}, \quad 0.38, \quad 0.87 \qquad \text{for } b_f = 0, 1, 3, 5.$$

Figure 2 (right) shows the full gradient, all paths included, in random LSTMs: with $b_f = 0$ it vanishes like a plain RNN, with $b_f = 3$ it stays of order one for a hundred steps. The full gradient exceeds the direct-path product ($\sigma(3)^{100} = 0.008$) because the indirect terms of (14), which contain the matrices W_q, add to it; for $b_f = 5$ they even make it grow slowly (7 at $k = 100$).

> **In physics — the cell is a leaky integrator with a learned time constant**
>
> With a constant forget gate f and input $u_t = i_t \tilde{c}_t$, the cell obeys $c_t = f c_{t-1} + u_t$. The Euler step (step 1) of $\tau \dot{c} = -c + \tau u(t)$ has this form with $f = 1 - 1/\tau$: an RC circuit, a low-pass filter, a damped mode. Matching the decay of the impulse response exactly, $f^k = e^{-k/\tau}$, gives the time constant
>
> $$\tau = -\frac{1}{\ln f} \approx \frac{1}{1 - f} \quad (f \to 1),$$
>
> so $f = 0.5$, 0.9 and 0.99 give memories of about 1.4, 9.5 and 99.5 steps. The forget-gate biases $b_f = 0, 1, 3, 5$ of the previous paragraph correspond to $\tau = 1.4$, 3.2, 21 and 149 steps. An LSTM is thus a bank of leaky integrators whose time constants are not fixed but set at every step by a learned function of the input and the state: a unit can hold a value for hundreds of steps and then reset it in one. Physicists who model a detector pulse or a slowly varying background with a filter of chosen time constant will recognise the structure; the LSTM chooses its own.

Variants. Many modifications have been tried. *Peephole connections* let the gates see the cell itself, adding terms $\mathbf{p}_q \odot \mathbf{c}_{t-1}$ (or $\mathbf{c}_t$ for the output gate) to the gate pre-activations [8]. *Coupled* input and forget gates set $\mathbf{i}_t = \mathbf{1} - \mathbf{f}_t$, so that writing and forgetting are tied. A large empirical study of eight variants on speech, handwriting and music found none clearly better than the standard LSTM; the forget gate and the squashing of the output were the most important components, while coupling the gates or removing the peepholes did not hurt [13].

6 Gated recurrent units

The *gated recurrent unit* (GRU) of Cho et al. [5] keeps the additive update of the LSTM with fewer parts: no separate cell, two gates instead of three.

$$\begin{aligned} \mathbf{z}_t &= \sigma\big(W_z\mathbf{h}_{t-1} + U_z\mathbf{x}_t + \mathbf{b}_z\big), \qquad \mathbf{r}_t = \sigma\big(W_r\mathbf{h}_{t-1} + U_r\mathbf{x}_t + \mathbf{b}_r\big), \\ \tilde{\mathbf{h}}_t &= \tanh\big(W_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + U_h\mathbf{x}_t + \mathbf{b}_h\big), \qquad \mathbf{h}_t = (\mathbf{1} - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \end{aligned} \tag{15}$$

The *update gate* $\mathbf{z}_t$ plays the roles of both the input gate and the forget gate of the LSTM, coupled as $\mathbf{i} = \mathbf{z}$, $\mathbf{f} = \mathbf{1} - \mathbf{z}$: the new state is a convex combination of the old state and the candidate, so it stays inside $[-1, 1]^d$. The *reset gate* $\mathbf{r}_t$ decides how much of the old state the candidate may use; with $\mathbf{r}_t = \mathbf{0}$ the candidate is computed from the input alone, as if the sequence had just started. The state is exposed directly, without an output gate. Along the direct path, $\partial\mathbf{h}_t/\partial\mathbf{h}_{t-1}$ contains the term $\mathrm{diag}(\mathbf{1} - \mathbf{z}_t)$, the counterpart of $\mathrm{diag}(\mathbf{f}_t)$ in (14). Here $\mathbf{z}_t$ is the update gate, not the logits of (4). Conventions differ: Cho et al. and PyTorch write $\mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t$, so their $\mathbf{z}_t$ is the fraction kept rather than the fraction written, and some libraries apply the reset gate after the product, $\mathbf{r}_t \odot (W_h\mathbf{h}_{t-1})$. The behaviour is similar.

Algorithm 6 One step of an LSTM and of a GRU (the affine maps of all gates are one matrix product)

Require: previous $\mathbf{h}_{t-1}$ (and $\mathbf{c}_{t-1}$ for the LSTM); input $\mathbf{x}_t$; stacked weights $W_\star \in \mathbb{R}^{4d\times d}$, $U_\star \in \mathbb{R}^{4d\times e}$, $\mathbf{b}_\star \in \mathbb{R}^{4d}$ (LSTM) or $3d$ rows (GRU)

function LSTMSTEP($\mathbf{h}_{t-1}, \mathbf{c}_{t-1}, \mathbf{x}_t$)
  $[\mathbf{a}^{(i)}; \mathbf{a}^{(f)}; \mathbf{a}^{(o)}; \mathbf{a}^{(c)}] \leftarrow W_\star\mathbf{h}_{t-1} + U_\star\mathbf{x}_t + \mathbf{b}_\star$ ▷ $O(4d(d+e))$, the whole cost
  $\mathbf{i} \leftarrow \sigma(\mathbf{a}^{(i)})$; $\mathbf{f} \leftarrow \sigma(\mathbf{a}^{(f)})$; $\mathbf{o} \leftarrow \sigma(\mathbf{a}^{(o)})$; $\tilde{\mathbf{c}} \leftarrow \tanh(\mathbf{a}^{(c)})$
  $\mathbf{c}_t \leftarrow \mathbf{f} \odot \mathbf{c}_{t-1} + \mathbf{i} \odot \tilde{\mathbf{c}}$; $\mathbf{h}_t \leftarrow \mathbf{o} \odot \tanh(\mathbf{c}_t)$ ▷ $O(d)$
  return $\mathbf{h}_t, \mathbf{c}_t$
end function
function GRUSTEP($\mathbf{h}_{t-1}, \mathbf{x}_t$)
  $[\mathbf{a}^{(z)}; \mathbf{a}^{(r)}] \leftarrow W_{zr}\mathbf{h}_{t-1} + U_{zr}\mathbf{x}_t + \mathbf{b}_{zr}$; $\mathbf{z} \leftarrow \sigma(\mathbf{a}^{(z)})$; $\mathbf{r} \leftarrow \sigma(\mathbf{a}^{(r)})$
  $\tilde{\mathbf{h}} \leftarrow \tanh\big(W_h(\mathbf{r} \odot \mathbf{h}_{t-1}) + U_h\mathbf{x}_t + \mathbf{b}_h\big)$ ▷ reset gate acts before W_h
  return $\mathbf{h}_t = (1 - \mathbf{z}) \odot \mathbf{h}_{t-1} + \mathbf{z} \odot \tilde{\mathbf{h}}$
end function

Running time — LSTM and GRU cells

Time: $O(4d(d+e))$ multiply–adds per step for the LSTM and $O(3d(d+e))$ for the GRU, against $O(d(d+e))$ for a plain RNN; the element-wise operations are $O(d)$. A sequence costs T times as much, and BPTT through the cell (by automatic differentiation) about twice the forward pass. *Memory*: $4d(d+e+1)$ parameters for the LSTM and $3d(d+e+1)$ for the GRU (libraries that keep two bias vectors per gate add $4d$ or $3d$); during training about six vectors of size d per step (gates, candidate, cell, state) must be stored. *Generation*: $O(4d(d+e))$ per token and a state of $2d$ numbers.

Worked example — 7. Counting parameters of recurrent cells

For hidden size $d = 512$ and input size $e = 256$, the cells have

plain RNN: 393 728 (Section 2.1), GRU: $3d(d+e+1) = 1\,181\,184$, LSTM: 1 574 912

parameters, the last being $4d(d+e+1)$; the forward cost per step is in the same ratio $1 : 3 : 4$. At equal parameter budget a GRU can therefore be about 15% wider than an LSTM ($\sqrt{4/3} \approx 1.15$ when $d \gg e$). Which is better depends on the task; with the forget-gate bias of Section 5, comparisons generally find the two close.

Modern connection — where gated recurrent networks are still used

Transformers (Note 15) have replaced recurrent networks in most of language processing, but gated RNNs remain a strong and economical choice wherever data arrive as a stream and the model must run with constant memory per step. In probabilistic forecasting, DeepAR trains one autoregressive LSTM on many related time series (the demand for thousands of products, for instance) and outputs a predictive distribution for each future value rather than a point forecast [32]. Small LSTMs and GRUs run on devices with tight memory, in streaming speech and sensor processing, and in control. Google's neural machine-translation system, deployed in 2016, was a stack of eight LSTM layers in the encoder and eight in the decoder, with residual connections and attention [41] — one of the last large recurrent systems before the transformer, and, with its attention, a bridge to Note 15.

7 Sequence-to-sequence learning

Many tasks map one sequence to another of different length with no position-by-position alignment: translating a sentence, summarising a document, transcribing speech, turning a question into an answer. The output sequence $w_{1:T'} = (w_1, \dots, w_{T'})$ cannot be predicted one label per input position, so the model is a *conditional* language model:

$$p(w_1, \dots, w_{T'} \mid \mathbf{x}_1, \dots, \mathbf{x}_T) = \prod_{t=1}^{T'} p(w_t \mid w_{<t}, \mathbf{x}_{1:T}). \tag{16}$$

The *encoder–decoder* architecture [5, 39] uses two recurrent networks. The *encoder* reads the input sequence and its final state becomes a fixed-size summary $\mathbf{s} = \mathbf{h}_T^{\text{enc}}$, the *context vector*. The *decoder* is an RNN language model whose initial state is $\mathbf{s}$ (or which receives $\mathbf{s}$ as an extra input at every step); it starts from `<bos>` and produces the output tokens one by one until `<eos>`. Figure 4 draws the arrangement. Several details were found to matter:

- the encoder and the decoder have *separate* parameters (they read different languages and do different jobs);
- stacking several LSTM layers in both helps substantially;
- Sutskever, Vinyals and Le found that reading the source sentence *in reverse order* improved their LSTM translation model markedly [39]: the first source words then sit close to the first target words in the unrolled graph, which shortens the paths the gradient must travel at the beginning of the output.

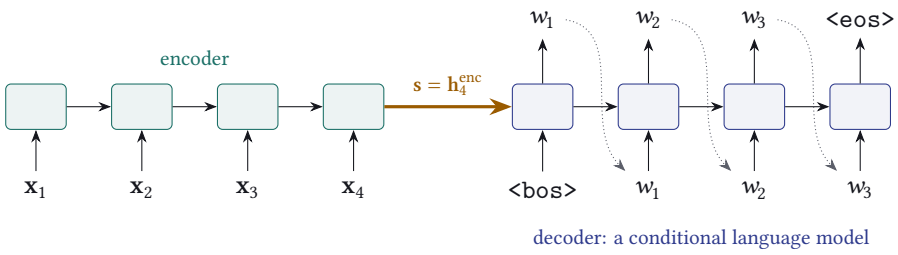


Figure 4. An encoder–decoder. The encoder (teal) reads the source $\mathbf{x}_1, \dots, \mathbf{x}_4$; its final state $\mathbf{s}$ is the only information about the source that reaches the decoder (indigo). The decoder's input at each step is the previous target token: the true one during training (teacher forcing), its own previous output at test time (dotted arrows).

Training with teacher forcing. The loss is the negative log of (16) for the reference output, summed over the training pairs. During training the decoder receives the *true* previous token w_{t-1} as input, not its own prediction: *teacher forcing*. The loss then decomposes into T' next-token cross-entropies that are all computed in one forward pass, and the gradient flows through decoder and encoder alike by BPTT. Sequences in a mini-batch have different lengths; they are padded to a common length with a padding token whose positions are excluded from the loss by a *mask*, and grouping sequences of similar length into the same mini-batch (*bucketing*) keeps the wasted computation small.

Exposure bias. At test time the true previous token is not available, and the decoder must consume its own outputs. It has never been trained on prefixes containing its own mistakes, so an early error can lead it into unfamiliar states where it makes more errors. This mismatch between training and use is called *exposure bias*. *Scheduled sampling* replaces, with a probability that increases during training, the true previous token by a token sampled from the model [3]; in practice teacher forcing remains the standard, because it is simple and stable and gives every target token's loss in one decoder pass (parallel over positions only for the attention-based decoders of Note 15; a recurrent decoder still loops over t).

Algorithm 7 One training step of an encoder–decoder with teacher forcing

Require: mini-batch of pairs $(\mathbf{x}^{(m)}_{1:T_m}, w^{(m)}_{1:T'_m})$, each $w^{(m)}$ ending with `<eos>`; encoder and decoder RNNs; output layer; optimiser

function Seq2SeqStep(batch)
  pad sources and targets to the longest lengths in the batch
  masks: $\mu^{(m)}_t = 1$ for real target tokens, 0 for padding
  for each pair m (in parallel) **do**
    $\mathbf{s} \leftarrow$ final state of the encoder on $\mathbf{x}^{(m)}$ ▷ Algorithm 2 without outputs
    $\mathbf{h} \leftarrow \mathbf{s}$; $w^{(m)}_0 \leftarrow$ `<bos>`; $\ell^{(m)} \leftarrow 0$
    for $t = 1, \dots, T'_{\max}$ **do**
      $\mathbf{h} \leftarrow \textsc{DecoderStep}(\mathbf{h}, \mathbf{e}(w^{(m)}_{t-1}))$ ▷ true previous token: teacher forcing
      $\ell^{(m)} \leftarrow \ell^{(m)} - \mu^{(m)}_t \log \operatorname{softmax}(V_{\text{out}}\mathbf{h} + \mathbf{b}_y)_{w^{(m)}_t}$
    end for
  end for
  $J \leftarrow \sum_m \ell^{(m)} \big/ \sum_{m,t} \mu^{(m)}_t$ ▷ mean loss per real target token
  BPTT through decoder and encoder; clip the gradient; optimiser step
end function

Running time — encoder–decoder training

Time per pair: $O\big(T\, c_{\text{cell}} + T'(c_{\text{cell}} + dV)\big)$ forward, with $c_{\text{cell}} = 4Ld(d+e)$ for an L-layer LSTM; backward about twice that. *Memory*: $O((T+T')Ld)$ stored states per pair. *Padding* wastes the fraction of masked positions; bucketing by length keeps it small. Teacher forcing lets all T' output losses be computed in one pass of the decoder; the encoder and decoder loops themselves remain sequential in time.

Worked example — 8. Where the parameters of a translation model are

Sutskever, Vinyals and Le used four LSTM layers with $d = 1000$ cells in both encoder and decoder, word embeddings of size $e = 1000$, a source vocabulary of 160 000 words and a target vocabulary of 80 000 words [39]. Count:

- embeddings: $(160\,000 + 80\,000) \times 1000 = 240\,000\,000$;

- LSTM layers: each has $4\times1000\times(1000+1000+1) = 8\,004\,000$; eight layers give 64 032 000;
- output softmax: $80\,000 \times 1001 = 80\,080\,000$.

Total 384 112 000, the 384 million parameters the authors report. Five-sixths of them sit in the vocabulary tables at the two ends; the recurrent layers that do the "translating" hold one-sixth. The whole meaning of a source sentence, however long, must nevertheless pass through one context vector of $4 \times 2 \times 1000 = 8000$ numbers (the states and cells of the four encoder layers).

The bottleneck. Everything the decoder learns about the source must pass through the fixed-size vector $\mathbf{s}$, whatever the length of the input. Letting the decoder consult all encoder states, with weights it computes itself, removes this bottleneck: this is *attention* (Note 15, Sections 1 and 2).

8 Decoding: greedy search, beam search and sampling

Once trained, a conditional language model defines a distribution over output sequences. For translation or transcription one wants the most probable output,

$$\hat{w}_{1:T'} = \arg\max_{w_{1:T'}} \sum_{t=1}^{T'} \log p(w_t \mid w_{<t}, \mathbf{x}), \tag{17}$$

but the number of candidates is $V^{T'}$, and the network's state after a prefix depends on the whole prefix, so dynamic programming does not apply. Exact search is out of reach; decoders approximate it.

Greedy decoding. Take the most probable token at every step and feed it back. This costs one decoder step per token but can be badly wrong: a token that looks best now may lead only to poor continuations.

Beam search. Keep the B best partial outputs (*hypotheses*), ranked by their log-probability so far. At each step, extend every hypothesis by every token, score the BV extensions, and keep the best B. A hypothesis that ends with `<eos>` is complete and keeps its score; the search stops when the B best hypotheses are all complete or a maximum length is reached, and returns the best complete one. With $B = 1$ beam search is greedy decoding; with $B = V^{T'}$ it would be exhaustive. Small beams (a handful of hypotheses) were enough for most of the gain in translation [39]. Because every token multiplies the probability by a number below one, scores favour short outputs; the usual correction ranks complete hypotheses by their log-probability divided by their length (or by a power of it).

Worked example — 9. Greedy versus beam search on a tiny vocabulary

The vocabulary is $\{a, b, \texttt{<eos>}\}$ and outputs have at most two words before `<eos>` (after two words, `<eos>` is certain). The decoder's distributions are

	a	b	`<eos>`
$p(w_1)$	0.5	0.4	0.1
$p(w_2 \mid w_1 = a)$	0.3	0.3	0.4
$p(w_2 \mid w_1 = b)$	0.1	0.1	0.8

Greedy: a (0.5), then `<eos>` (0.4): output "a" with probability $0.5 \times 0.4 = 0.20$. *Beam*, $B = 2$: step 1 keeps a ($\log 0.5 = -0.693$) and b (-0.916); the complete hypothesis "`<eos>`" (0.1) falls out.

Step 2 scores the six extensions:

$$aa\ 0.15, \quad ab\ 0.15, \quad a\ \texttt{<eos>}\ 0.20, \quad ba\ 0.04, \quad bb\ 0.04, \quad b\ \texttt{<eos>}\ 0.32.$$

The best two are b <eos> ($\log 0.32 = -1.139$) and a <eos> (-1.609), both complete, so the search stops and returns "b" with probability 0.32. Enumerating all seven possible outputs (b: 0.32; a: 0.20; aa, ab: 0.15 each, since <eos> is then certain; the empty output: 0.10; ba, bb: 0.04 each) confirms that b is the most probable: greedy search was misled by the attractive first token. Here normalising by length (two tokens for b <eos> and a <eos>, three for aa <eos>) does not change the ranking: -0.570 for b <eos> against -0.632 for aa <eos> and -0.805 for a <eos>.

Algorithm 8 Beam search decoding

Require: decoder state $\mathbf{h}_0$ (e.g. the encoder's context $\mathbf{s}$); beam width B; maximum length $T_{\max}$

function BeamSearch($\mathbf{h}_0$)
  $\mathscr{H} \leftarrow \{(\text{prefix} = (\texttt{<bos>}),\ \text{score} = 0,\ \text{state} = \mathbf{h}_0)\}$; $\mathscr{C} \leftarrow \emptyset$ ▷ live and complete hypotheses
  for $t = 1, \dots, T_{\max}$ **do**
    $\mathscr{N} \leftarrow \emptyset$
    for each $(\pi, s, \mathbf{h}) \in \mathscr{H}$ **do**
      $\mathbf{h}' \leftarrow$ DecoderStep($\mathbf{h}$, $\mathbf{e}$(last token of π)) ▷ $O(c_{\text{cell}})$
      $\mathbf{q} \leftarrow \log \text{softmax}(V_{\text{out}}\mathbf{h}' + \mathbf{b}_y)$ ▷ $O(dV)$
      add $(\pi \cdot k,\ s + q_k,\ \mathbf{h}')$ to $\mathscr{N}$ for every token k ▷ or only the top B tokens
    end for
    $\mathscr{K} \leftarrow$ the B highest-scoring elements of $\mathscr{N} \cup \mathscr{C}$
    $\mathscr{C} \leftarrow$ the elements of $\mathscr{K}$ ending in <eos>; $\mathscr{H} \leftarrow$ the others
    if $\mathscr{H} = \emptyset$ **then break** ▷ the B best are all complete
    end if
  end for
  return the best element of $\mathscr{C}$ (of $\mathscr{H}$ if $\mathscr{C} = \emptyset$) ▷ score, or score per token
end function

Running time — beam search

Time: $O\big(T_{\max}B(c_{\text{cell}} + dV)\big)$ for the decoder steps and softmaxes, plus $O(T_{\max}BV)$ for finding the best extensions (a partial selection, not a full sort, of the BV candidates; taking each hypothesis's own top B tokens first, $O(V)$ each, leaves a final merge of only B^2 candidates). The cost is B times that of greedy decoding, and the B hypotheses are processed as one mini-batch. *Memory*: B decoder states and B prefixes, $O(B(Ld + T_{\max}))$. Beam search is approximate and not even monotone in B: a larger beam usually, but not always, finds a higher-scoring output, and in translation very large beams sometimes produce *worse* outputs (too short or generic), a sign that the model's most probable sequence is not always the best one.

Sampling. When many outputs are acceptable (a story, a dialogue reply), one samples instead of maximising, by the ancestral sampling of Section 2. Three modifications reshape each next-token distribution before the draw. The *temperature* T_{temp} of Note 2, modern box of Section 2.2 (written with a subscript here to keep T for lengths), divides the logits, $p_k \mapsto p_k^{1/T_{\text{temp}}} / \sum_j p_j^{1/T_{\text{temp}}}$: values below one sharpen the distribution, values above one flatten it. *Top-k* sampling keeps only the k most probable tokens and renormalises. *Nucleus* (top-p) sampling keeps the smallest set of most probable tokens whose probabilities add up to at least p, and renormalises [18]. For the first-step distribution of Worked example 9, $(0.5, 0.4, 0.1)$: temperature 0.5 gives $(0.595, 0.381, 0.024)$ and temperature 2

gives $(0.427, 0.382, 0.191)$; top-2 sampling and nucleus sampling with $p = 0.8$ both keep $\{a, b\}$ and sample them with probabilities $(0.556, 0.444)$.

Algorithm 9 Autoregressive generation with temperature, top-k and nucleus sampling

Require: decoder state $\mathbf{h}_0$; temperature $T_{\text{temp}} > 0$; optional k (top-k) or p (nucleus); maximum length $T_{\max}$

function SAMPLE($\mathbf{h}_0$)
  $\mathbf{h} \leftarrow \mathbf{h}_0$; $w \leftarrow$ <bos>; output $\leftarrow$ empty list
  for $t = 1, \dots, T_{\max}$ **do**
    $\mathbf{h} \leftarrow \text{DECODERSTEP}(\mathbf{h}, \mathbf{e}(w))$ ▷ $O(c_{\text{cell}})$
    $\mathbf{z} \leftarrow (V_{\text{out}}\mathbf{h} + \mathbf{b}_y)/T_{\text{temp}}$ ▷ $O(dV)$
    if top-k: set all but the k largest entries of $\mathbf{z}$ to $-\infty$ ▷ partial selection, $O(V)$
    $\boldsymbol{\pi} \leftarrow \text{softmax}(\mathbf{z})$
    if nucleus: sort $\boldsymbol{\pi}$, keep the shortest prefix of mass $\geq p$, renormalise ▷ $O(V \log V)$
    draw $w \sim \boldsymbol{\pi}$; append w to the output ▷ Note 2, Section 2.2
    if $w =$ <eos> **then break**
    end if
  end for
  return the output
end function

Running time — sampling

Time: $O\big(T'(c_{\text{cell}} + dV)\big)$ for an output of T' tokens: one decoder step and one softmax per token, as for greedy decoding; top-k adds $O(T'V)$ for the partial selection, nucleus sampling $O(T'V \log V)$ for the sort (or less with a partial sort). *Memory*: the current state, $O(Ld)$ numbers, plus the T' output tokens. The steps are strictly sequential, and independent samples can be drawn in parallel as a mini-batch.

Modern connection — decoding a large language model

Beam search remains the decoder of choice when there is essentially one right answer — translation, speech recognition. For open-ended generation it fails in a characteristic way: the most probable continuations of a text are bland and repetitive, and maximising probability drives a model into loops. Holtzman et al. documented this degeneration and proposed nucleus sampling as the remedy [18]: it cuts off the unreliable tail of the distribution, whose many individually improbable tokens together carry much probability, while keeping the variety of the head. All of these controls act only on the output distribution of one step, so they apply unchanged to recurrent and transformer decoders.

9 Sequence classification and bidirectional networks

Sequence classification. When a whole sequence receives one label — the sentiment of a review, the class of an event recorded as a time series, the type of a particle from the sequence of its hits — the network must turn a variable-length sequence into a fixed-size vector and classify it. The simplest summary is the final state $\mathbf{h}_T$, as for the encoder of Section 7. Formally this is a sequence-to-sequence problem whose output has a single token, the label, so the encoder of Section 7 followed by a softmax is enough. The final state is biased towards the end of the sequence, and averaging or taking the element-wise maximum of all states, $\frac{1}{T}\sum_t \mathbf{h}_t$ or $\max_t \mathbf{h}_t$, often works better for long inputs. A softmax layer (Note 11, Section 5) on this summary gives the class probabilities, and BPTT distributes the gradient of the single loss over all time steps.

Bidirectional RNNs. When the whole input is available before any output is needed — tagging a sentence, labelling the frames of a recording, encoding the source of a translation — there is no reason to label position t from positions $1, \dots, t$ alone (Section 2.1): the word after an ambiguous word often decides its meaning. A *bidirectional* RNN [34] runs two independent RNNs, one forward over $\mathbf{x}_1, \dots, \mathbf{x}_T$ and one backward over $\mathbf{x}_T, \dots, \mathbf{x}_1$, and represents position t by the concatenation of their states:

$$\overrightarrow{\mathbf{h}}_t = \text{cell}_{\rightarrow}\big(\overrightarrow{\mathbf{h}}_{t-1}, \mathbf{x}_t\big), \qquad \overleftarrow{\mathbf{h}}_t = \text{cell}_{\leftarrow}\big(\overleftarrow{\mathbf{h}}_{t+1}, \mathbf{x}_t\big), \qquad \mathbf{h}_t = \big[\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t\big] \in \mathbb{R}^{2d}. \tag{18}$$

Now $\mathbf{h}_t$ depends on the entire sequence and is centred on position t. Bidirectional layers can be stacked (the input of layer l is the concatenated $\mathbf{h}_t^{(l-1)}$), and deep bidirectional LSTMs were for several years widely used for speech recognition [12] and for tagging. They are unusable when the future is not available: a language model that generates text, or any system that must answer before the input ends (streaming speech, on-line triggers), must stay causal.

Algorithm 10 A bidirectional RNN tagger: forward pass

Require: inputs $\mathbf{x}_1, \dots, \mathbf{x}_T$; forward and backward cells (separate parameters); output layer for K labels

function BiRNNTag($\mathbf{x}_{1:T}$)
  $\overrightarrow{\mathbf{h}}_0 \leftarrow \mathbf{0}$; **for** $t = 1, \dots, T$: $\overrightarrow{\mathbf{h}}_t \leftarrow \text{cell}_{\rightarrow}(\overrightarrow{\mathbf{h}}_{t-1}, \mathbf{x}_t)$
  $\overleftarrow{\mathbf{h}}_{T+1} \leftarrow \mathbf{0}$; **for** $t = T, \dots, 1$: $\overleftarrow{\mathbf{h}}_t \leftarrow \text{cell}_{\leftarrow}(\overleftarrow{\mathbf{h}}_{t+1}, \mathbf{x}_t)$ ▷ independent of the first loop
  for $t = 1, \dots, T$ **do**
    $\mathbf{y}_t \leftarrow \text{softmax}\left(V_{\text{out}}[\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] + \mathbf{b}_y\right)$ ▷ $O(2dK)$
  end for
  return $\mathbf{y}_1, \dots, \mathbf{y}_T$ ▷ loss: $-\sum_t \log y_{t,\, y_t^\star}$, gradients by BPTT in both directions
end function

Running time — bidirectional RNN

Time: twice the cost of a unidirectional layer, $O(2T\, c_{\text{cell}})$, plus $O(TdK)$ for the outputs; the two directions are independent and can run in parallel. *Memory*: twice the parameters of one direction (V_{out} reads $2d$ numbers) and $O(2Td)$ stored states. *Latency*: no output can be produced before the whole input has been read, because $\overleftarrow{\mathbf{h}}_1$ depends on $\mathbf{x}_T$.

Modern connection — from bidirectional LSTMs to bidirectional transformers

Deep bidirectional LSTM language models trained on large corpora produced word representations that depend on the whole sentence (ELMo [28]): the vector of "charge" differs between "electric charge" and "charge the battery". Adding these contextual vectors to existing models improved many language-understanding tasks at once, an early instance of pre-training a large sequence model and reusing its representations. The idea was soon carried over to transformers, whose attention reads both directions at once (BERT, Note 15, Section 6.1); generative models, by contrast, stay causal, reading left to right only.

10 Recursive networks on trees

Sentences have structure beyond order: in "hardly a clean signal" the word "hardly" modifies the whole phrase "a clean signal", not the next word alone. A *recursive* neural network [36] follows a tree instead of a chain. Given a binary tree — for a sentence, its parse tree from a syntactic parser — each leaf is a word embedding, and each internal node computes its vector from its two children

with the same composition function,

$$\mathbf{p} = \tanh\big(W[\mathbf{a};\mathbf{b}] + \mathbf{b}\big), \qquad W \in \mathbb{R}^{d\times 2d}, \tag{19}$$

until the root represents the whole phrase. A classifier can read any node, so every phrase can be labelled, for example with its sentiment. The weights are shared over all nodes, as an RNN shares them over time; an RNN is the special case of a completely left-branching tree. Gradients flow down the tree by backpropagation through structure, and the depth of the graph is the depth of the tree, $O(\log n)$ for a balanced tree of n words instead of n for a chain — shorter paths for the gradient. Socher et al. trained such models on a treebank in which every phrase of thousands of movie-review sentences carries a sentiment label, and found that a richer composition with a bilinear tensor term ($\mathbf{p} = \tanh(\mathbf{v}^{\mathsf{T}}\mathscr{T}^{[1:d]}\mathbf{v} + W\mathbf{v} + \mathbf{b})$ with $\mathbf{v} = [\mathbf{a};\mathbf{b}]$, one slice $\mathscr{T}^{[k]} \in \mathbb{R}^{2d\times 2d}$ per output component) captured negation and contrast better than (19) [37]. The need for a parser and the difficulty of batching trees of different shapes limited their use; attention (Note 15) later learned soft structures without a parser.

Algorithm 11 Recursive network: bottom-up composition over a binary tree

Require: binary tree with n leaves (words $w_1, \dots, w_n$); embedding matrix E; composition weights W, $\mathbf{b}$; node classifier V_{out}

```
function COMPOSE(node)
    if node is a leaf for word w then return e(w)
    end if
    a ← COMPOSE(left child); b ← COMPOSE(right child)        ▷ post-order
    p ← tanh(W[a; b] + b); store y_node ← softmax(V_out p) if labelled   ▷ O(2d^2)
    return p
end function
```

Running time — recursive network

Time: a binary tree with n leaves has $n-1$ internal nodes, so the forward pass costs $O(nd^2)$, plus $O(nd^3)$ with the tensor term (one $2d \times 2d$ quadratic form per output component); backward about twice that. *Memory*: $O(nd)$ stored node vectors, $2d^2 + d$ composition parameters. Nodes at the same depth are independent and can be batched, so the sequential depth is the height of the tree, between $\log_2 n$ and $n-1$.

11 Recurrence and its limits today

A recurrent network compresses an arbitrarily long past into a state of fixed size, updates it at constant cost per step, and needs only that state to continue. These are real strengths for streams and for generation. They come with three limits, each visible in this note:

1. *Sequential computation.* Step t needs step $t-1$, so training cannot be parallelised over time (runtime box of Algorithm 2). On hardware built for large matrix products this leaves most of the machine idle for long sequences; a causal convolution (Note 13, Section 9) or attention processes all positions of a training sequence at once.
2. *Long paths.* Information from k steps back has passed through k applications of the cell; gating keeps the gradient alive but does not shorten the path (Section 5).
3. *A fixed-size bottleneck.* Everything the model knows about the past must fit in $\mathbf{h}_t$ (Section 7; Note 15, Section 1).

Attention addresses the last two directly (Note 15, Sections 2 and 5). Its cost grows with the square of the sequence length, and generation must keep all past keys and values (Note 15, Section 6.2). These costs have brought recurrence back.

Modern connection — the return of linear recurrences: state-space models

A *linear* recurrence $\mathbf{h}_t = A\mathbf{h}_{t-1} + B\mathbf{x}_t$, $\mathbf{y}_t = C\mathbf{h}_t$ (here B and C are matrices, not the beam width) unrolls into $\mathbf{y}_t = \sum_{k\geq 0} CA^k B\,\mathbf{x}_{t-k}$: a causal convolution with the kernel $(CB, CAB, CA^2B, \dots)$. A model built from such layers can therefore be trained like a convolutional network, in parallel over the sequence (with the fast Fourier transform, or with a parallel prefix scan, since composing linear maps is associative: a *prefix scan* computes all partial compositions, and thus all $\mathbf{h}_1, \dots, \mathbf{h}_T$, in $O(\log T)$ parallel rounds by combining neighbouring pairs, then pairs of pairs), and run like an RNN at generation time, with a state of fixed size and constant cost per token. Structured state-space models (S4) chose A from a continuous-time system designed to remember the input's history and handled sequences of more than ten thousand steps (16 384 in the Path-X benchmark) [14]; a linear recurrent unit with a diagonal, complex A whose eigenvalues are kept inside the unit circle achieves similar results, showing that the key was stable linear dynamics plus non-linear layers between them [25]. Mamba makes the recurrence *selective*: the step size and the matrices B and C depend on the current input, so the model can decide what to store and what to ignore, much as an LSTM's gates do, while a hardware-aware scan keeps training parallel [15]. The vanishing-gradient analysis of Section 4 applies to them directly: the eigenvalues of A set the memory time scales, and they are parameterised so that $|\lambda| < 1$ always holds, as close to one as long memories require.

In physics — reservoir computing: train only the read-out

An extreme answer to the difficulty of training recurrent weights is not to train them. An *echo state network* uses a large, fixed, random recurrent network — the *reservoir* — with its recurrent matrix scaled to a spectral radius below about one, so that the state is a fading memory of the input (the "echo state property", a negative Lyapunov exponent in the language of Section 4). Only the linear read-out $\mathbf{y}_t = V_{\text{out}}\mathbf{h}_t$ is trained, by least squares (Note 8), which is fast and convex [19]. Reservoirs predict chaotic time series remarkably well: Pathak et al. forecast the spatiotemporally chaotic Kuramoto–Sivashinsky equation for several Lyapunov times from data alone [27]. The same idea is used with physical reservoirs — optical, spintronic or mechanical systems whose natural dynamics provide the random recurrent map.

Costs at a glance. Table 1 collects the costs of this note per sequence of length T, for hidden width d, input width e, vocabulary V, L layers and beam width B (one layer unless stated).

Summary A language model factorises the probability of a sequence into next-token distributions; counting n-grams fails on unseen contexts, and a neural model with learned embeddings shares strength between similar ones. A recurrent network replaces the fixed window by a hidden state $\mathbf{h}_t = \tanh(W\mathbf{h}_{t-1} + U\mathbf{x}_t + \mathbf{b})$ updated with shared weights, and is trained by back-propagation through time, in which every shared parameter sums one gradient term per step; truncated BPTT bounds the memory for long streams. The gradient from k steps back is a product of k Jacobians D_sW, bounded by $(\gamma\,\|W\|)^k$ and growing at the rate of the largest Lyapunov exponent: it vanishes or explodes exponentially. Clipping tames explosion; orthogonal initialisation helps at the start; the LSTM's additive cell, with forget gates initialised open, gives the gradient a path that only the gates attenuate, and the GRU does the same with two gates.

Table 1. Costs of the models and algorithms of this note (multiply–adds; one sequence).

operation	time	parameters	memory (training)
neural n-gram LM, per token	$O(d(n-1)e + dV)$	$Ve + d((n-1)e+1) + V(d+1)$	$O(d+V)$
RNN, forward	$O(T(Ld^2 + de + dV))$	$d(d+e+1)$ first layer, $d(2d+1)$ above	$O(TLd)$
BPTT (full)	$\approx 2\times$ forward	—	$O(TLd)$ states
truncated BPTT, chunks k	as BPTT, $\lceil T/k \rceil$ updates	—	$O(kLd)$
LSTM / GRU step	$O(4d(d+e))$ / $O(3d(d+e))$	$4d(d+e+1)$ / $3d(d+e+1)$	$\approx 6d$ per step
orthogonal initialisation	$O(d^3)$ once	—	$O(d^2)$
encoder–decoder	$O((T+T')c_{\text{cell}} + T'dV)$	encoder + decoder + output	$O((T+T')Ld)$
greedy decoding	$O(T'(c_{\text{cell}} + dV))$	—	$O(Ld)$
beam search, width B	$O(T'B(c_{\text{cell}} + dV))$	—	$O(B(Ld+T'))$
sampling (top-k / top-p)	$O(T'(c_{\text{cell}} + dV))$ + selection	—	$O(Ld+T')$
bidirectional layer	$O(2Tc_{\text{cell}})$	$2\times$ one direction	$O(2Td)$
recursive net, n leaves	$O(nd^2)$	$2d^2 + d$	$O(nd)$

Encoder–decoders map sequences to sequences, are trained with teacher forcing and decoded by greedy or beam search, or sampled; bidirectional RNNs use both directions when the whole input is known, and recursive networks follow trees. The fixed-size state is both the strength and the bottleneck of recurrence, which attention (Note 15) removes, while linear state-space models bring recurrence back for long sequences.

Questions and exercises

Questions are sorted by kind. Parts A and B are of the kind used in the midterm examination: they test understanding, interpretation and short calculations (at most simple derivatives), not long proofs. Parts C and D go further and are recommended for practice and for the project.

A. Conceptual questions

A1. Write the chain-rule factorisation of the probability of a sentence. Which approximation does an n-gram model add, and why does counting fail for $n = 5$ even with a very large corpus?

A2. Explain why the tokens `<bos>` and `<eos>` are needed in a language model. What would go wrong without `<eos>`?

A3. In the neural language model of Bengio et al., how do learned embeddings allow the model to assign a sensible probability to a context it has never seen? Where is most of its computation spent, and why?

A4. Explain what "unrolling" a recurrent network means. Why does the number of parameters not depend on the length of the sequence, and what is the analogue of this property in a convolutional network?

A5. An RNN language model has no Markov assumption, yet in practice it does not use information from arbitrarily far back. Explain both halves of this statement.

A6. What changes when an RNN is used for sequence labelling instead of language modelling? Give two sequence-labelling tasks. Why might stacking recurrent layers help, and what restriction does a left-to-right labeller have?

A7. In backpropagation through time, why is the gradient of W a sum over time steps? Through which two paths does the hidden state $\mathbf{h}_t$ influence the loss?

A8. What does truncated BPTT truncate, and what does it keep? Can a model trained with chunks of 50 steps use information from 200 steps back? Can it learn to?

A9. State the bound on $\|\partial \mathbf{h}_T / \partial \mathbf{h}_t\|$ for a tanh RNN. Which condition is sufficient for vanishing gradients and which is necessary for exploding gradients? Why is the problem more severe in time than in depth?

A10. Why can the spectral norm of W be a misleading guide to the long-run behaviour of W^k? Which quantity decides it?

A11. Explain the connection between vanishing or exploding gradients and the Lyapunov exponent of the network's dynamics. What does a positive exponent mean for the forward dynamics and for training?

A12. What does gradient clipping do, why does it help against exploding gradients, and why does it not help against vanishing ones?

A13. Why does an orthogonal recurrent matrix preserve the norm of a vector? Why does a tanh RNN with orthogonal W still suffer from vanishing gradients (Figure 2)?

A14. Describe the roles of the input, forget and output gates of an LSTM. What happens to the cell when $\mathbf{f}_t = \mathbf{1}$ and $\mathbf{i}_t = \mathbf{0}$?

A15. Explain why the additive cell update of the LSTM helps gradients flow over long time gaps. Which factor replaces the product $D_s W$ along the direct path?

A16. Why is the forget-gate bias usually initialised to 1 or more? Interpret the choice as a time constant.

A17. Compare the GRU with the LSTM: which gates correspond, what is missing, and what does the reset gate do?

A18. What is teacher forcing, why is it convenient, and what is exposure bias?

A19. Explain the bottleneck of the basic encoder–decoder. Why did reversing the source sentence help?

A20. Why is exact maximisation of $p(w_{1:T'} \mid \mathbf{x}_{1:T})$ over output sequences intractable? Explain how beam search approximates it and why its scores favour short outputs.

A21. Why is beam search a good decoder for translation but a poor one for story generation? Describe nucleus sampling.

A22. A recurrent network must classify a whole sequence. Compare using the final state $\mathbf{h}_T$ with averaging or max-pooling all states $\mathbf{h}_t$. Why can the final state be a poor summary of a long sequence, and in what sense is sequence classification a special case of sequence-to-sequence learning?

A23. When can a bidirectional RNN be used and when not? Give one task of each kind.

A24. How does a recursive network differ from a recurrent one? What does it need as input besides the words?

A25. Name three limitations of recurrent networks that motivate attention, and one advantage of recurrence that state-space models exploit.

B. Short calculations

B1. From the corpus "<bos> the jet is wide .", "<bos> the jet is narrow .", "<bos> the muon is narrow .", estimate the bigram probabilities $p(\text{jet} \mid \text{the})$, $p(\text{wide} \mid \text{is})$ and $p(\text{narrow} \mid \text{is})$. Compute the probability and the perplexity of "the muon is wide ." (five predicted tokens). Give a grammatical sentence over the same vocabulary that receives probability zero and name the bigram responsible (hint: consider a sentence in which "is" is followed by "the"). Recompute that bigram's probability with a pseudo-count of 1 over the 7 predictable tokens.

B2. A scalar RNN has $h_t = \tanh(w h_{t-1} + u x_t)$ with $w = 1$, $u = 0.5$, $h_0 = 0$ and inputs $x = (2, -2)$. Compute h_1 and h_2. With the loss $\ell = \frac{1}{2} h_2^2$, compute $\partial \ell / \partial w$ and $\partial \ell / \partial u$ by BPTT, and check $\partial \ell / \partial w$ by writing ℓ as an explicit function of w and differentiating.

B3. Count the parameters of: (a) a plain RNN language model with $V = 50\,000$, $e = 256$, $d = 512$ (embedding table, cell and output layer); (b) the same model with an LSTM cell; (c) the LSTM model with tied input and output embeddings (take $e = d = 512$). Which part dominates in each case?

B4. For the linear recurrence $\mathbf{h}_t = A \mathbf{h}_{t-1}$ with $A = \begin{pmatrix} 0.9 & 0 \\ 0 & 1.05 \end{pmatrix}$, compute A^{20} and the norms of $A^{20}\mathbf{u}$ for $\mathbf{u} = (1, 0)$, $(0, 1)$ and $(1, 1)/\sqrt{2}$. After how many steps does the second component exceed 10 times its initial value? What do the eigenvalues tell you about vanishing and exploding gradients in this network?

B5. One LSTM unit has $c_{t-1} = -0.5$ and gate pre-activations $a_i = 1$, $a_f = 0$, $a_o = 2$, $a_c = -1$. Compute i_t, f_t, o_t, $\tilde{c}_t$, c_t and h_t. Then compute one GRU step for a unit with $h_{t-1} = 0.4$, $z_t = 0.25$ and candidate pre-activation 1 (so that $\tilde{h}_t = \tanh 1$). Which of the quantities c_t (LSTM cell), h_t (LSTM output) and h_t (GRU state) can take values outside $[-1, 1]$, and why?

B6. An LSTM's forget gates stay at $f = 0.95$. After how many steps has the direct-path gradient fallen below 10^{-2}? Which forget-gate bias, with all other pre-activation contributions zero, gives $f = 0.95$? What is the time constant?

B7. Run beam search with $B = 1$, $B = 2$ and $B = 3$ on the following decoder: $p(w_1)$: *a* 0.4, *b* 0.35, *c* 0.25; after *a*: <eos> 0.3, otherwise uniform over a, b, c; after *b*: <eos> 0.6, *a* 0.4; after *c*: <eos> 1; at most two words, after which <eos> is certain. Give the output and its probability in each case, and the true most probable output.

B8. Apply temperatures $T_{\text{temp}} = 0.5$ and $T_{\text{temp}} = 2$, top-2 sampling and nucleus sampling with $p = 0.9$ to the next-token distribution $(0.6, 0.25, 0.1, 0.05)$. Give the resulting four distributions.

C. Derivations

C1. Derive the BPTT recursion (8) and the gradients (9) for the RNN (4) with softmax output and cross-entropy loss, by drawing the unrolled graph and applying the chain rule. Extend the

result to a two-layer RNN (7).

C2. Show that $\|AB\| \le \|A\|\,\|B\|$ for the spectral norm and deduce (11). For a diagonalisable W in the linear case, show that $\|W^k \mathbf{u}\|$ grows or decays like $\rho(W)^k$ for a generic $\mathbf{u}$. Give an example where $\|W\| > 1$ but $W^k \to 0$.

C3. Derive the Jacobian $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1}$ of the LSTM (14), writing out the indirect terms through each gate. Show that with constant gates the cell is the solution of a linear first-order recursion and give its impulse response and time constant.

C4. For the GRU (15), derive $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1}$ and identify the term that plays the role of the forget gate. Show that if $\mathbf{z}_t$ and $\mathbf{r}_t$ are constant and $\tilde{\mathbf{h}}_t$ does not depend on $\mathbf{h}_{t-1}$, the state is an exponential moving average of the candidates.

C5. A linear state-space layer has $h_t = \lambda h_{t-1} + b x_t$, $y_t = c h_t$ (scalars, $|\lambda| < 1$, $h_0 = 0$). Show that $y_t = \sum_{k=0}^{t-1} c \lambda^k b \, x_{t-k}$. Write one step as the affine map $h \mapsto \alpha h + \beta_t$ with $\alpha = \lambda$, $\beta_t = b x_t$; show that applying (α_1, β_1) and then (α_2, β_2) gives the affine map $(\alpha_2 \alpha_1,\ \alpha_2 \beta_1 + \beta_2)$, and that this combination rule is associative (so that a parallel prefix scan applies). Compute $\partial y_T / \partial x_{T-k}$ and relate λ to a memory time.

D. Programming

D1. Implement Algorithms 2 and 3 in numpy for a one-layer character-level RNN language model. Check the gradients by central differences on a tiny network (Note 12, Section 4). Train it with truncated BPTT and clipping (Algorithm 4) on a few hundred kilobytes of text, report the validation perplexity per character, and sample text (Algorithm 9) at temperatures 0.5, 1 and 1.5.

D2. In PyTorch, compare a tanh RNN, a GRU and an LSTM on the following memory task: the input is a sequence of random bits ± 1 of length $k + 1$, and the target, given at the last step, is the first bit. For $k = 5, 10, 20, 50, 100$ and a fixed budget of training steps, report the test accuracy of each model; repeat the LSTM with forget-gate bias 0 and 3. Plot the gradient norm $\|\partial \ell / \partial \mathbf{h}_1\|$ at initialisation against k.

D3. Reproduce Figure 3 (right): for random tanh networks of width 64 (as in the figure) and then 200, and gains $\sigma_w \in [0.5, 3]$, estimate the largest Lyapunov exponent by iterating a tangent vector with renormalisation (physics box of Section 4), with and without random input. How does the crossing point move with the width? Then train echo state networks (fixed reservoir, least-squares read-out) to predict the Lorenz system ($\dot{x} = 10(y - x)$, $\dot{y} = x(28 - z) - y$, $\dot{z} = xy - \tfrac{8}{3} z$, integrated with step 0.01) one step ahead and compare the forecast horizon, in Lyapunov times, for reservoir spectral radii 0.5, 0.9 and 1.2.

D4. Implement an encoder–decoder with GRUs that learns to reverse sequences of digits of length 5 to 15, trained with teacher forcing. Implement greedy decoding and Algorithm 8, and compare the sequence accuracy of $B = 1$, 3 and 10 as a function of the sequence length. Does accuracy drop for the longest sequences? Relate the answer to the bottleneck of Section 7.

Further reading. Goodfellow, Bengio and Courville [9], chapter 10, covers this note's material in depth; Murphy [24], chapter 15, treats neural networks for sequences together with attention; Graves [11] is a readable account of generating sequences with LSTMs; Pascanu, Mikolov and Bengio [26] give the analysis of exploding and vanishing gradients with the dynamical-systems view.

References

[1] Y. Bengio, R. Ducharme, P. Vincent and C. Jauvin, A neural probabilistic language model, *J. Mach. Learn. Res.* 3, 1137–1155 (2003).

[2] Y. Bengio, P. Simard and P. Frasconi, Learning long-term dependencies with gradient descent is difficult, *IEEE Trans. Neural Netw.* 5, 157–166 (1994).

[3] S. Bengio, O. Vinyals, N. Jaitly and N. Shazeer, Scheduled sampling for sequence prediction with recurrent neural networks, *NeurIPS* (2015).

[4] R. T. Q. Chen, Y. Rubanova, J. Bettencourt and D. Duvenaud, Neural ordinary differential equations, *NeurIPS* (2018).

[5] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk and Y. Bengio, Learning phrase representations using RNN encoder–decoder for statistical machine translation, *EMNLP* (2014).

[6] J. L. Elman, Finding structure in time, *Cogn. Sci.* 14, 179–211 (1990).

[7] F. A. Gers, J. Schmidhuber and F. Cummins, Learning to forget: continual prediction with LSTM, *Neural Comput.* 12, 2451–2471 (2000).

[8] F. A. Gers and J. Schmidhuber, Recurrent nets that time and count, *Proc. IJCNN* (2000).

[9] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press (2016).

[10] A. Graves, M. Liwicki, S. Fernández, R. Bertolami, H. Bunke and J. Schmidhuber, A novel connectionist system for unconstrained handwriting recognition, *IEEE Trans. Pattern Anal. Mach. Intell.* 31, 855–868 (2009).

[11] A. Graves, Generating sequences with recurrent neural networks, arXiv:1308.0850 (2013).

[12] A. Graves, A. Mohamed and G. Hinton, Speech recognition with deep recurrent neural networks, *ICASSP* (2013).

[13] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink and J. Schmidhuber, LSTM: a search space odyssey, *IEEE Trans. Neural Netw. Learn. Syst.* 28, 2222–2232 (2017).

[14] A. Gu, K. Goel and C. Ré, Efficiently modeling long sequences with structured state spaces, *ICLR* (2022).

[15] A. Gu and T. Dao, Mamba: linear-time sequence modeling with selective state spaces, arXiv:2312.00752 (2023).

[16] M. Hibat-Allah, M. Ganahl, L. E. Hayward, R. G. Melko and J. Carrasquilla, Recurrent neural network wave functions, *Phys. Rev. Research* 2, 023358 (2020).

[17] S. Hochreiter and J. Schmidhuber, Long short-term memory, *Neural Comput.* 9, 1735–1780 (1997).

[18] A. Holtzman, J. Buys, L. Du, M. Forbes and Y. Choi, The curious case of neural text degeneration, *ICLR* (2020).

[19] H. Jaeger and H. Haas, Harnessing nonlinearity: predicting chaotic systems and saving energy in wireless communication, *Science* 304, 78–80 (2004).

[20] R. Jozefowicz, W. Zaremba and I. Sutskever, An empirical exploration of recurrent network architectures, *ICML* (2015).

[21] Q. V. Le, N. Jaitly and G. E. Hinton, A simple way to initialize recurrent networks of rectified linear units, arXiv:1504.00941 (2015).

[22] T. Mikolov, M. Karafiát, L. Burget, J. Černocký and S. Khudanpur, Recurrent neural network based language model, *Proc. Interspeech* (2010).

[23] F. Morin and Y. Bengio, Hierarchical probabilistic neural network language model, *AISTATS* (2005).

[24] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press (2022).

[25] A. Orvieto, S. L. Smith, A. Gu, A. Fernando, C. Gulcehre, R. Pascanu and S. De, Resurrecting recurrent neural networks for long sequences, *ICML* (2023).

[26] R. Pascanu, T. Mikolov and Y. Bengio, On the difficulty of training recurrent neural networks, *ICML* (2013).

[27] J. Pathak, B. Hunt, M. Girvan, Z. Lu and E. Ott, Model-free prediction of large spatiotemporally chaotic systems from data: a reservoir computing approach, *Phys. Rev. Lett.* 120, 024102 (2018).

[28] M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee and L. Zettlemoyer, Deep contextualized word representations, *NAACL* (2018).

[29] O. Press and L. Wolf, Using the output embedding to improve language models, *EACL* (2017).

[30] K. Rajan, L. F. Abbott and H. Sompolinsky, Stimulus-dependent suppression of chaos in recurrent neural networks, *Phys. Rev. E* 82, 011903 (2010).

[31] D. E. Rumelhart, G. E. Hinton and R. J. Williams, Learning representations by back-propagating errors, *Nature* 323, 533–536 (1986).

[32] D. Salinas, V. Flunkert, J. Gasthaus and T. Januschowski, DeepAR: probabilistic forecasting with autoregressive recurrent networks, *Int. J. Forecast.* 36, 1181–1191 (2020).

[33] A. M. Saxe, J. L. McClelland and S. Ganguli, Exact solutions to the nonlinear dynamics of learning in deep linear neural networks, *ICLR* (2014).

[34] M. Schuster and K. K. Paliwal, Bidirectional recurrent neural networks, *IEEE Trans. Signal Process.* 45, 2673–2681 (1997).

[35] R. Sennrich, B. Haddow and A. Birch, Neural machine translation of rare words with subword units, *ACL* (2016).

[36] R. Socher, C. C.-Y. Lin, A. Y. Ng and C. D. Manning, Parsing natural scenes and natural language with recursive neural networks, *ICML* (2011).

[37] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Y. Ng and C. Potts, Recursive deep models for semantic compositionality over a sentiment treebank, *EMNLP* (2013).

[38] H. Sompolinsky, A. Crisanti and H. J. Sommers, Chaos in random neural networks, *Phys. Rev. Lett.* 61, 259–262 (1988).

[39] I. Sutskever, O. Vinyals and Q. V. Le, Sequence to sequence learning with neural networks, *NeurIPS* (2014).

[40] P. J. Werbos, Backpropagation through time: what it does and how to do it, *Proc. IEEE* 78, 1550–1560 (1990).

[41] Y. Wu *et al.*, Google's neural machine translation system: bridging the gap between human and machine translation, arXiv:1609.08144 (2016).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.

The IFT 6390 slides on recurrent neural networks are based on material by Hugo Larochelle.